Data Communication and Computer Networks

Data Link Layer

Dr. RAVI B , Asst. Prof , CSE Dept.

Data Link Layer (DLL)

1. Design issues of DLL
2. Error detection and correction
3. Elementary data link protocols
4. Sliding window protocols.

Design issues of DLL

Design issues of DLL

- **Services Provided to the Network Layer**
- **Framing**
- **Error Control**
- **Flow Control**

Design issues of DLL

- The data link layer takes the packets it gets from the network layer and encapsulates them into frames for transmission.

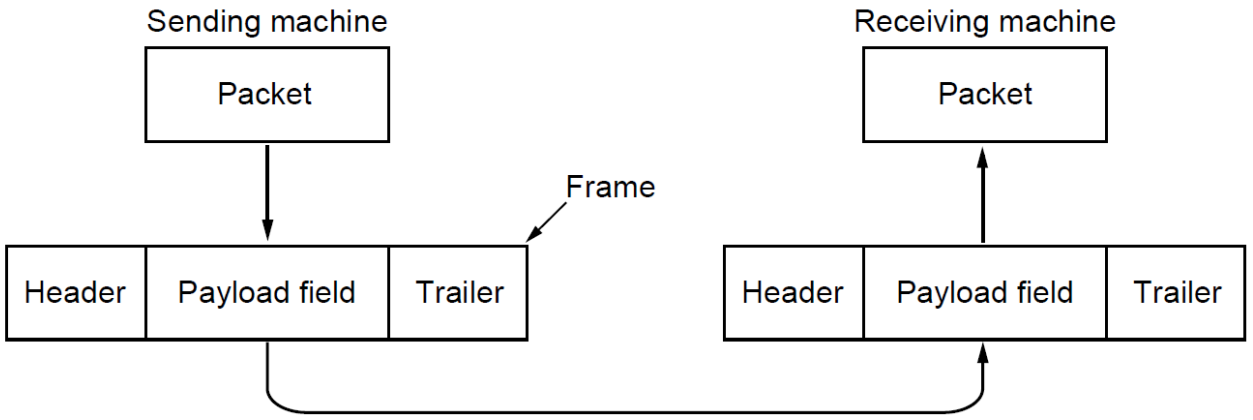
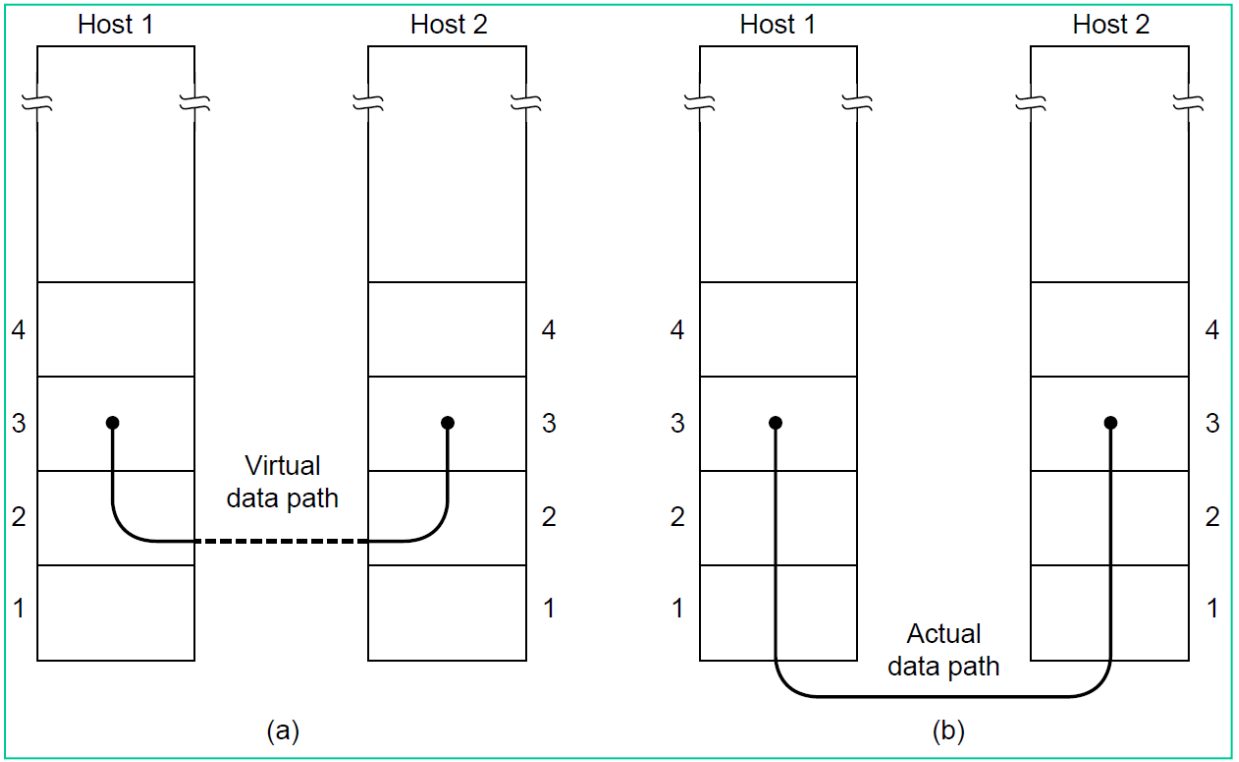


Fig. 3-1. Relationship between packets and frames.

1. Services Provided to the Network Layer Framing

- The job of the data link layer is to transmit the bits (given network layer process) to the destination machine so they can be handed over to the network layer there, as shown in Fig



1. Services Provided to the Network Layer Framing

- The data link layer can be designed to offer various services to network layer.

1. **Unacknowledged connectionless service.**
2. **Acknowledged connectionless service.**
3. **Acknowledged connection-oriented service.**

1. Services Provided to the Network Layer Framing

1. **Unacknowledged connectionless service.**
   - The source machine sends independent frames to the destination machine **without having the destination machine acknowledge them.**
   - **No logical connection** is established beforehand or released afterward.
   - If a frame is lost due to noise on the line, **no attempt is made to detect the loss or recover** from it in the data link layer.

1. Services Provided to the Network Layer Framing

1. **Unacknowledged connectionless service.**
   - Appropriate when the error rate is very low
   - Appropriate for real-time traffic, such as voice, in which late data are worse than bad data

1. Services Provided to the Network Layer Framing

2. Acknowledged connectionless service.

- No logical connections used, but each frame sent is individually acknowledged.
- Allows retransmission of the unacknowledged data.
- useful over unreliable channels, such as wireless systems.

1. Services Provided to the Network Layer Framing

2. Acknowledged connection-oriented service.

- Establish a connection before any data are transferred.
- Each frame sent over the connection is numbered, and the data link layer guarantees that each frame sent is indeed received.
- Furthermore, it guarantees that each frame is received exactly once and that all frames are received in the right order.

2. Framing

- Framing refers to breaking the network layer bits stream to discrete frames by the data link layer; These frames are then given to the physical layer for the transmission.
- It is up to the data link layer at the receiver side to detect and, if necessary, correct errors.
- There are several approaches for the data link layer to break the bit stream up into discrete frames.

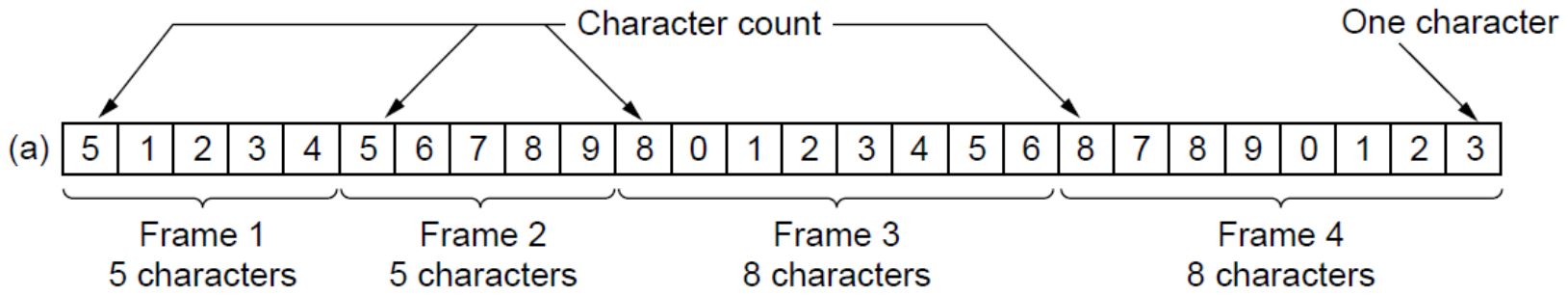
2. Framing

We look at 4 methods of framing.

1. **Character count.**
2. **Flag bytes with byte stuffing.**
3. **Starting and ending flags, with bit stuffing.**
4. **Physical layer coding violations**

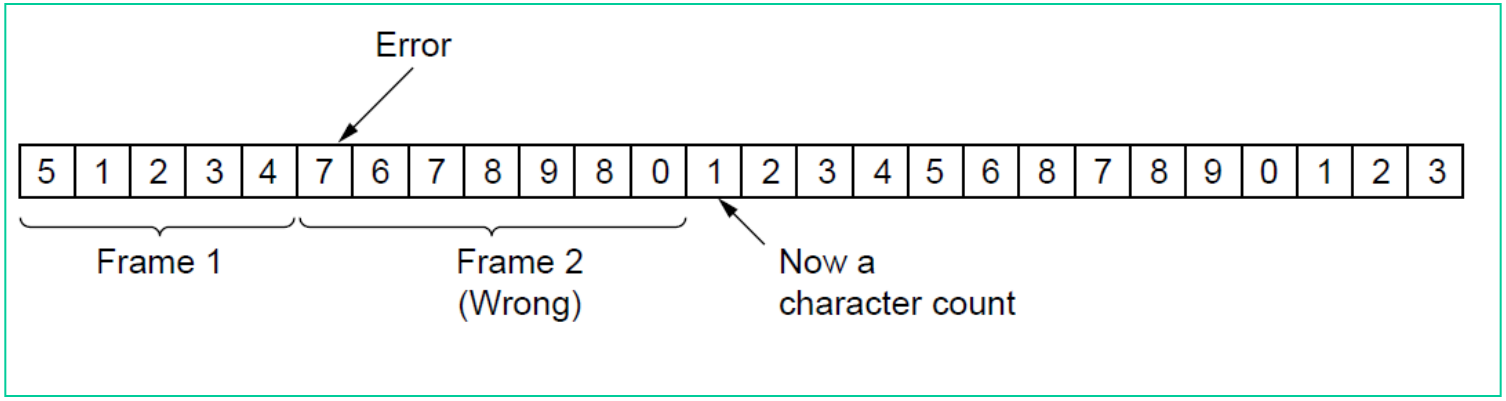
2. Framing

- **Character count** uses a field in the header to specify the number of characters in the frame.
- When the data link layer at the destination sees the character count, it knows how many characters follow and hence where the end of the frame is.
- This technique is shown in Figfor four frames of sizes 5, 5, 8, and 8 characters, respectively.
-



2. Framing

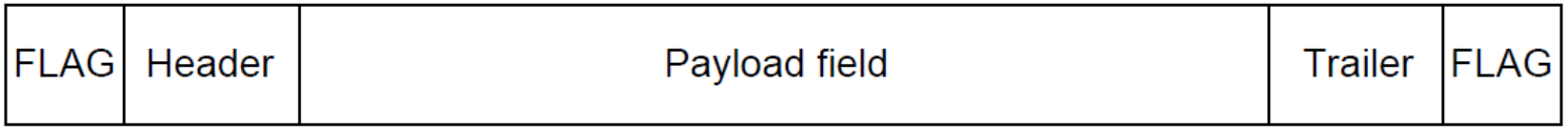
- The trouble with Character count is that the count can be garbled by a transmission error.
- In that case destination will get out of synchronization and will be unable to locate the start of the next frame.



- For this reason, the character count method is rarely used anymore

2. Framing

- To overcome this problem, the second uses a special byte, which is called a **flag byte**, to mark start and end of each frame. Two consecutive flag bytes indicate the end of one frame and start of the next one.

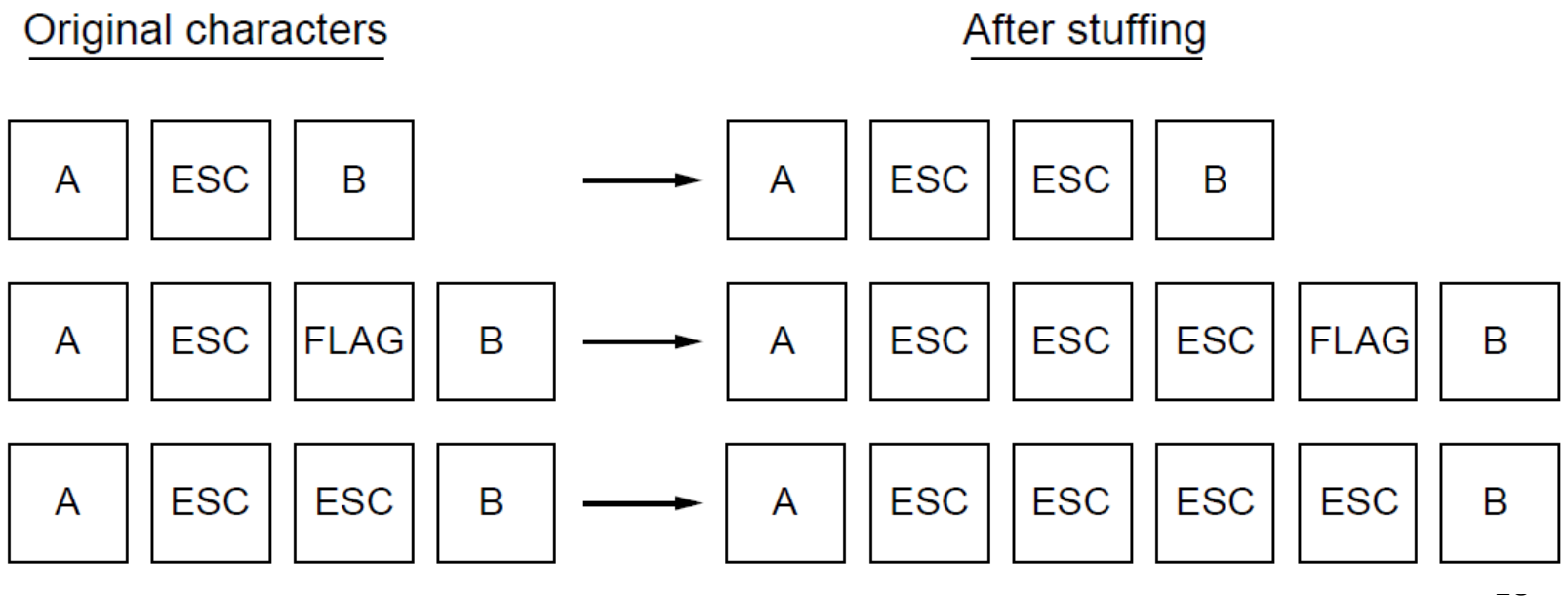


2. Framing

- It may easily happen that the flag byte's **bit pattern occurs in the data**. One way to solve this problem is to have the sender's data link layer insert a special escape byte (ESC) just before each "accidental" flag byte in the data. The data link layer on the receiving end removes the escape byte before the data are given to the network layer. This technique is called byte stuffing or character stuffing.

2. Framing

- What happens if an escape byte occurs in the middle of the data? The answer is that it, too, is stuffed with an escape byte.
- Thus, any single escape byte is part of an escape sequence, whereas a doubled one indicates that a single escape occurred naturally in the data.

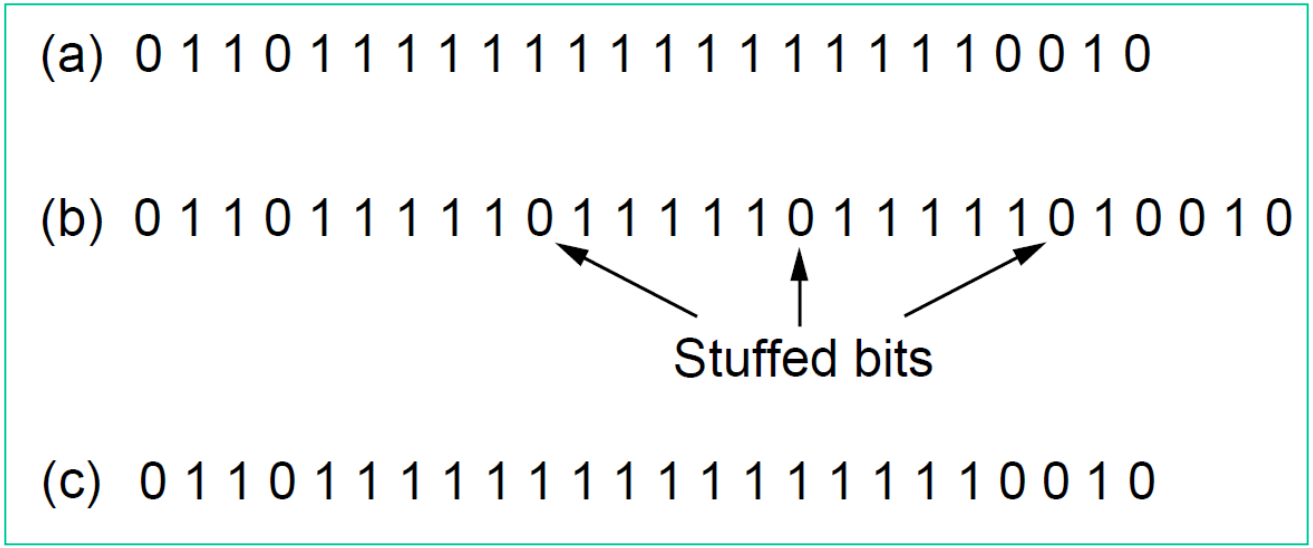


2. Framing

- A major disadvantage of using **byte stuffing or character stuffing** method is that it is closely tied to the use of 8-bit characters. Not all character codes use 8-bit characters. For example. UNICODE uses 16-bit characters

2. Framing

- The new technique ,called bit stuffing, allows each frame to begin and end with a special bit pattern, 01111110 (in fact, a flag byte).
- Whenever the sender's data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit into the outgoing bit stream. This bit stuffing is analogous to byte stuffing.



2. Framing

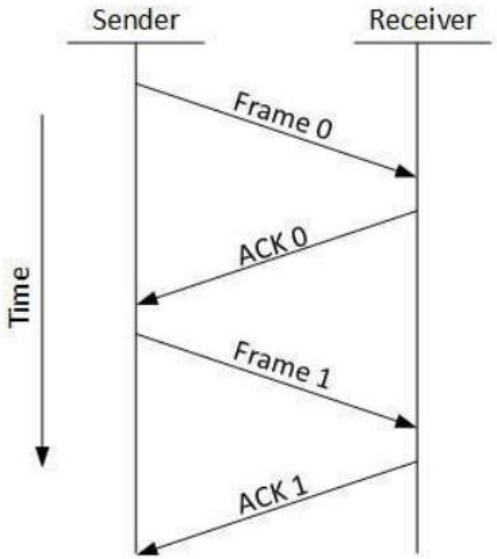
Advantages of bit stuffing:

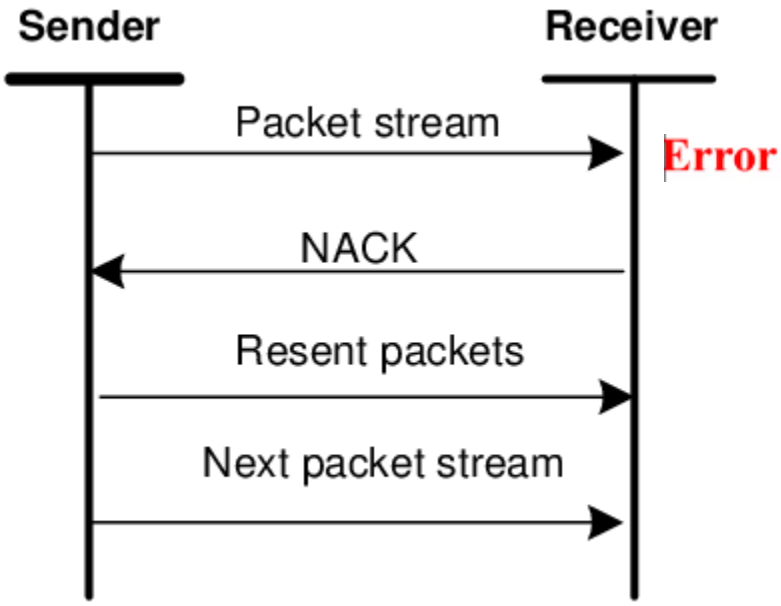
- With bit stuffing, the boundary between two frames can be unambiguously recognized by the flag pattern.

- Thus, if the receiver loses track of where it is, all it has to do is scan the input for flag sequences, since they can only occur at frame boundaries and never within the data.

3. Error Control

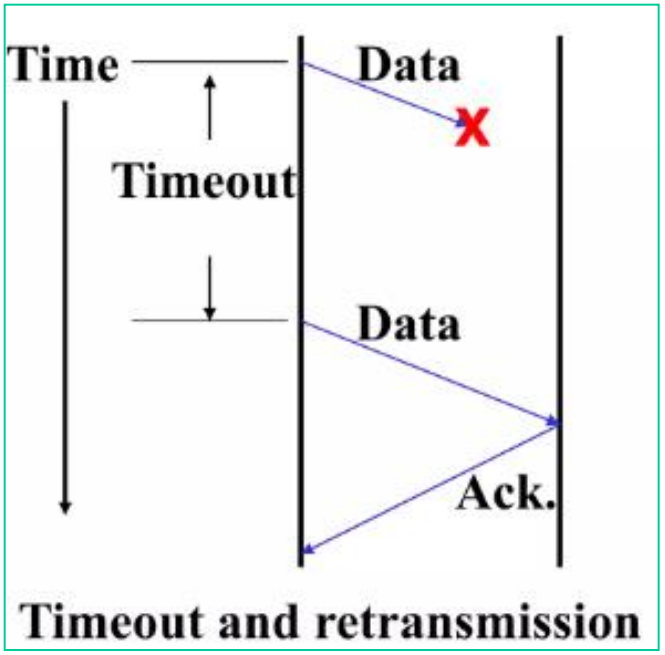
- The usual way to ensure reliable data delivery is to send +ve ACK if the receiver has received the frames without an error. A –ve ACK is sent if the frame is received with an error. In this case the frame must be transmitted again.



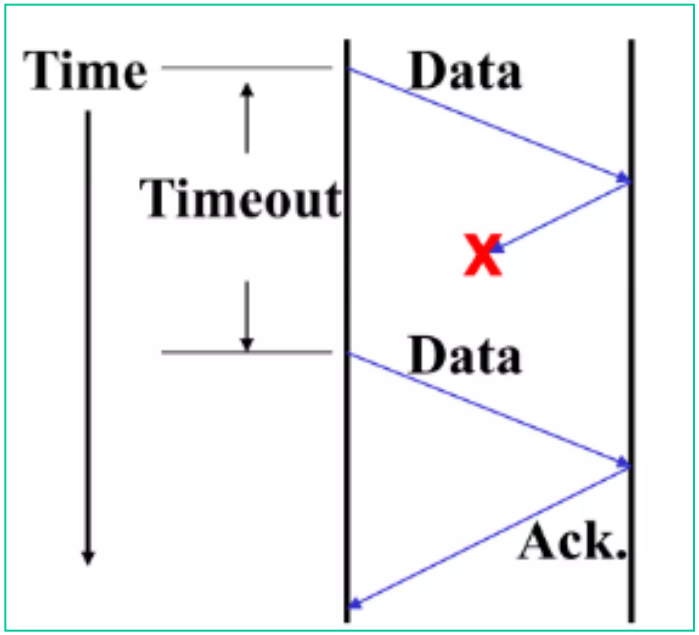


3. Error Control

- When the sender transmits a frame, it generally also starts a timer. The sender waits for an ACK until the timer expires. When the timer expires, the sender transmits the frame again. This means either frame is lost or ACK is lost.



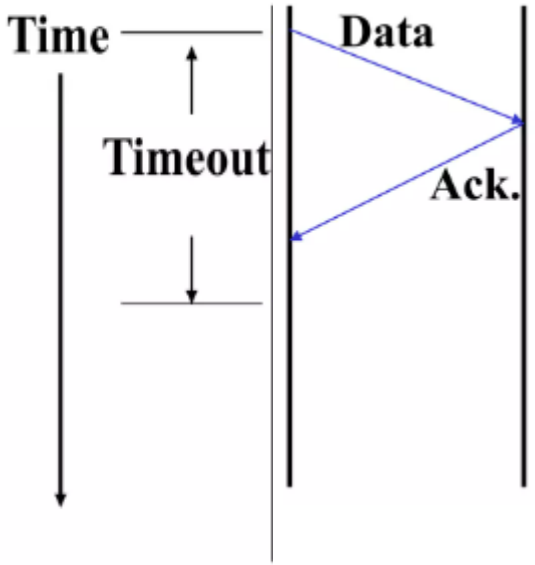
Timeout and retransmission



Timeout and retransmission

3. Error Control

- If the frame is correctly received and the acknowledgement gets back before the timer runs out, then the timer will be canceled.

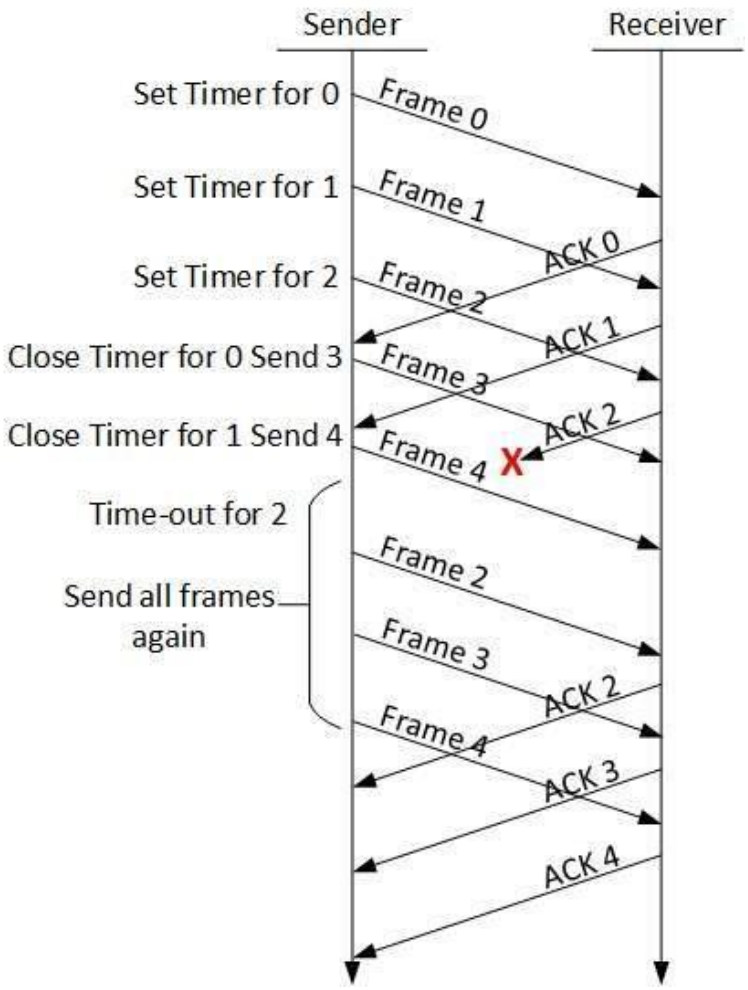


Timer will be canceled in this scenario

3. Error Control

- When frames may be transmitted multiple times there is a danger that the receiver will accept the same frame two or more times and pass it to the network layer more than once.

- The solution is to assign sequence numbers to outgoing frames, so that the receiver can distinguish retransmissions from originals



3. Error Control

- The whole issue of managing the timers and sequence numbers so as to ensure that each frame is ultimately passed to the network layer at the destination exactly once, no more and no less, is an important part of the data link layer's duties.

4. Flow Control

- Flow controls deals with sender transmitting frames faster than the receiver can accept them.

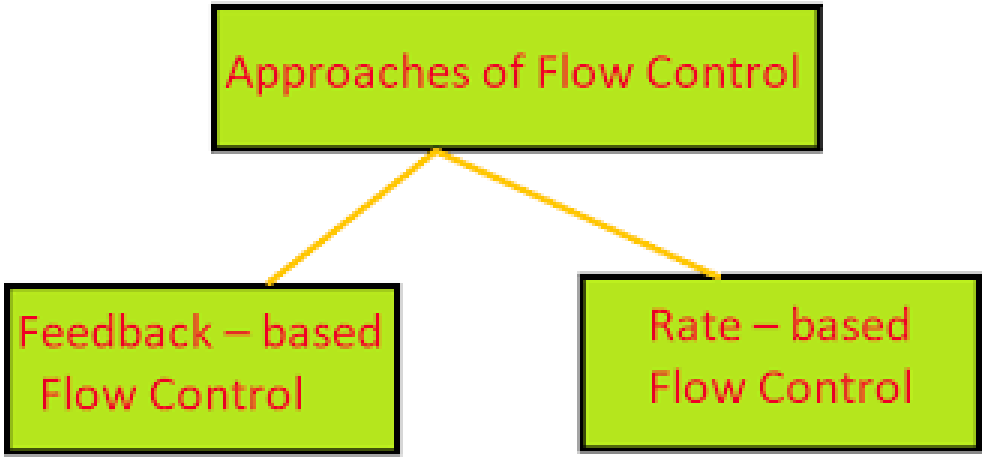
- Two approaches are commonly used.
  1. **feedback-based flow control**,
  2. **rate-based flow control**

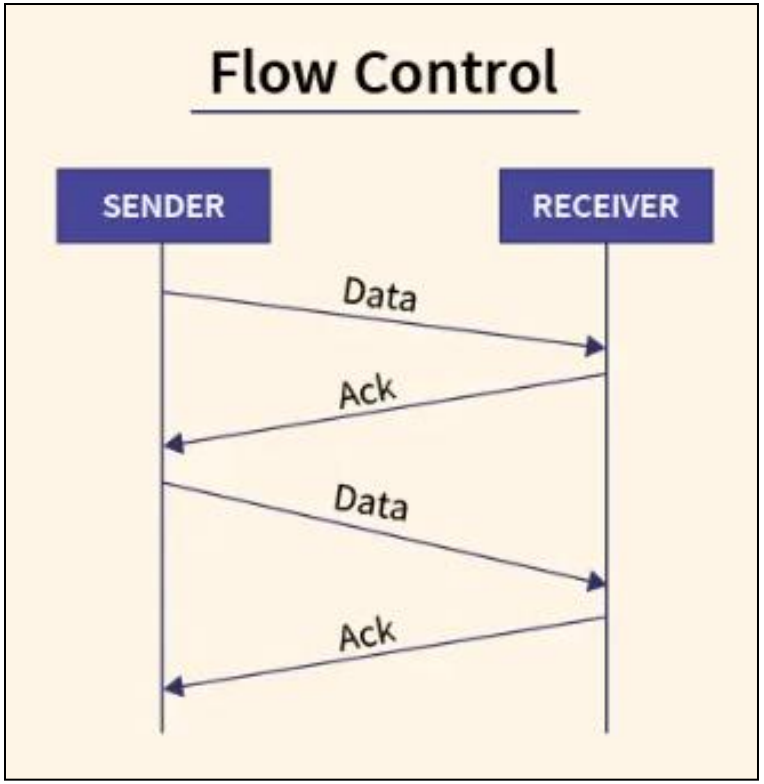
- In the first method the receiver sends back information to the sender giving it permission to send more/less data or at least telling the sender how the receiver is doing.

- In the Second method, the protocol has a built-in mechanism that limits the rate at which senders may transmit data, without using feedback from the receiver.

4. Flow Control

- Flow controls





Feedback-based flow control

Error Detection and Correction

Error Detection and Correction

1. **Error-Correcting Codes**
2. **Error-Detecting Codes**

Error Detection and Correction

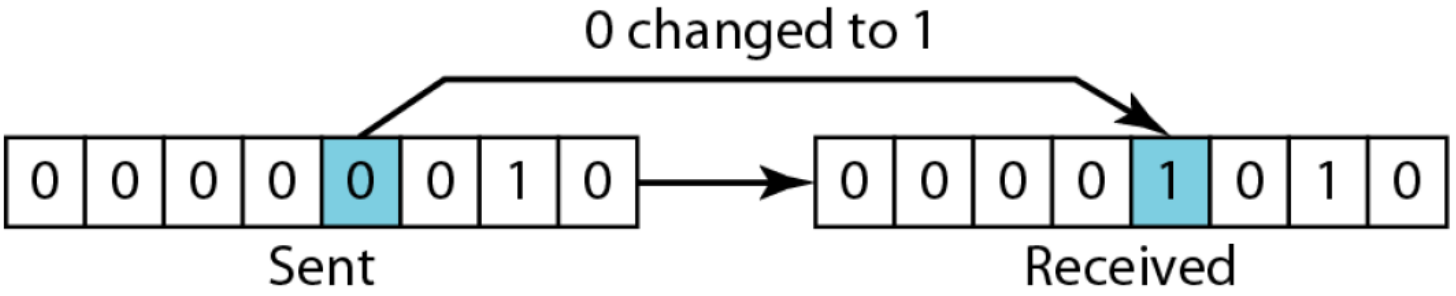
Types of error

1. Single-bit error
2. Burst error

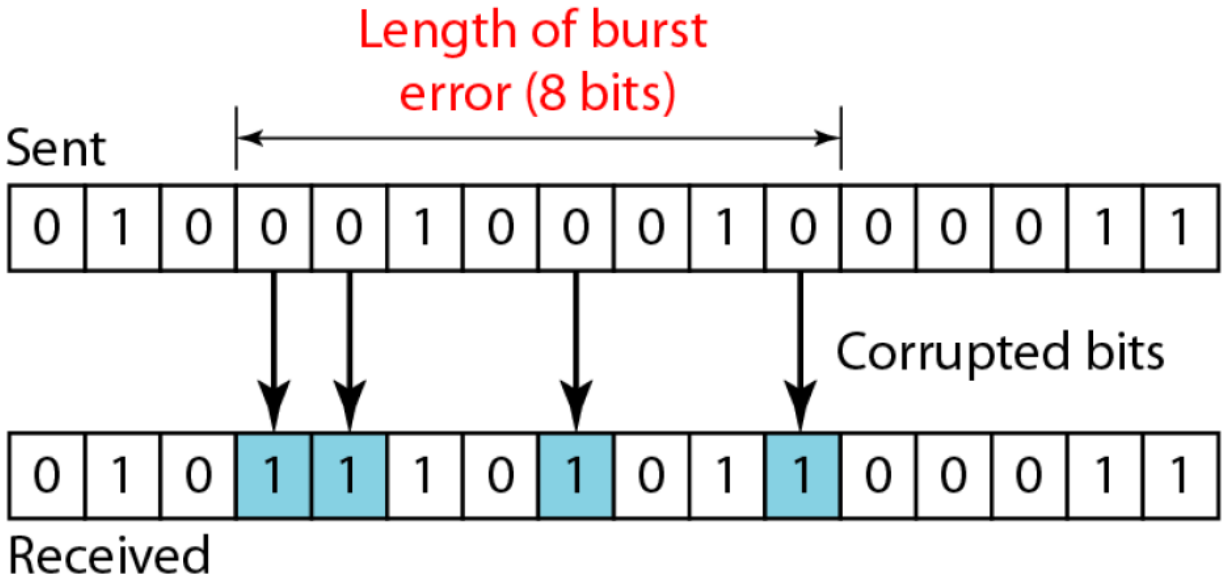
1. In a **single-bit error**, only 1 bit in the data unit has changed
2. A **burst error** means that 2 or more bits in the data unit have changed

Error Detection and Correction

Single-bit error



Burst error



Error-Correcting Codes

- Two basic strategies for dealing with errors:
  1. **error-correcting codes**
  2. **error-detecting codes**

- In the first method, sender includes enough redundant information along with each block of data sent, to enable the receiver to correct the errors. This is also known as **forward error correction**

- Second method includes enough redundancy to allow the receiver to deduce that an error occurred, and request a retransmission.

Error-Correcting Codes

- Normally, a frame consists of m data bits and r redundant, or check bits.
- Total length of the frame is **n = m + r**
- An n-bit unit containing data and check bits is often referred to as **an n-bit codeword.**

- **For any two codewords, it is possible to determine how many corresponding bits differ using XOR.**

```
10001001
10110001
00111000
```

Differ in 3 bits

Error-Correcting Codes

- The number of bit positions in which two codewords differ is called the **Hamming distance, which is the fundamental concept of error detection and correction.**
- The smallest Hamming distance between all possible codewords in set of codewords is **minimum Hamming distance .**
- What is the minimum hamming distance for codrwords **010, 011, 101** and **111**?

- d(010, 011) = 1, d(010, 101) = 3, d(010, 111) = 2.
- d(011, 101) = 2, d(011, 111) = 1, d(011, 111) = 1

Hence, the Minimum Hamming Distance, d_{min} = 1.

Error-Correcting Codes

Note :

1. To detect upto *s* error in a set of codewords , the minimum hamming distance must be s +1. That is $d_{min} = s+1$.

2. To correct upto *t* errors in a set of codewords, the minimum hamming distance must be 2t+1. That is $d_{min} = 2t+1$

Error-Correcting Codes

- Upto how many errors can be detected and corrected in the following set of code words.

Codewords
00000
01011
10101
11110

- We can see that $d_{min} = 3$
- Thus we can detect up to 2 errors.
- $d_{min} = 2t+1 = 3$, **Thus t = 1.**

  Thus we can correct 1 bit error**.**

Error-Correcting Codes

Hamming code:

- The Hamming code system was developed by **Richard Hamming** in the 1950s. It is commonly used in error correction code (ECC) RAM.
- d_{min} for hamming code is 3. So it can detect upto 2 bit errors and correct single-bit errors.
-

Error-Correcting Codes

Hamming code:

- A Hamming code is denoted by ***H(n,k)*** , *where n is the codeword size and k is the dataword size.* ***n-k*** *bits are extra bits added to the dataword. These bits are called* ***parity bits****.*
- *Example : H(11,7) means 11 is the codeword size and 7 is the dataword size. 11-7 =4 are parity bit size*

Error-Correcting Codes

Hamming code:

- A parity bit is used to make number of 1's in a specific set of data bits even or odd. (even or odd parity). We use even parity for this example.
- The value of r is the least possible integer such that binary representation of (k+r) has r bits.

Thus, $2^{r-1} \leq k + r \leq 2^r - 1$

$$k + r \leq 2^r - 1$$

$$k \leq 2^r - r - 1$$

Error-Correcting Codes

Hamming code:

Let k = 7. What is r?

r	$2^r - r - 1$
1	0
2	1
3	4
4	11 > k

Thus r = 4 so that 4 bits are required to represent 11.

Error-Correcting Codes

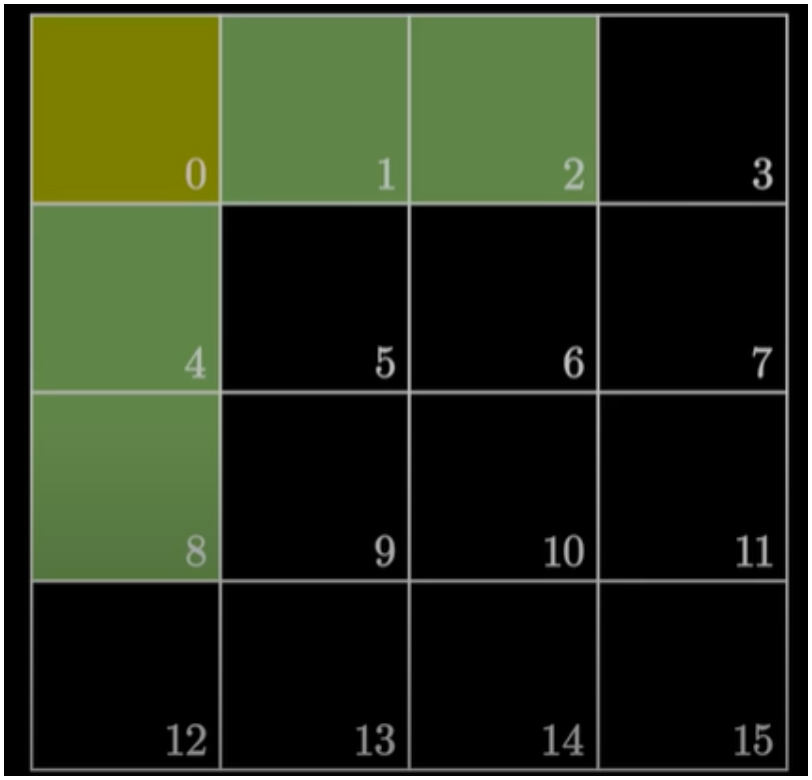
Hamming code:

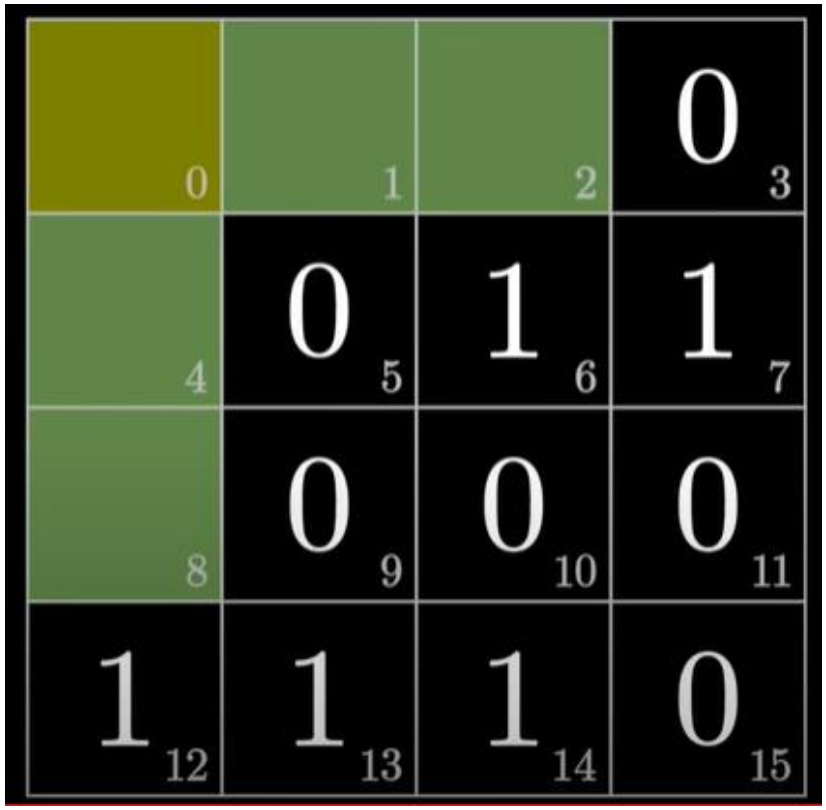
- So we need add 4 parity bits to the data bits to get codeword for the transmission.
- Each parity is responsible for detecting 1 or more errors in in the received codeword.
- The hamming code can detect up to 2 errors and correct 1 error.

Error-Correcting Codes

Hamming code:

- Let d = 00110001110, so k=11 and r= 4 , ***H(15,11***)
- Divide k+r = 15 bits into a block of 4 bits.



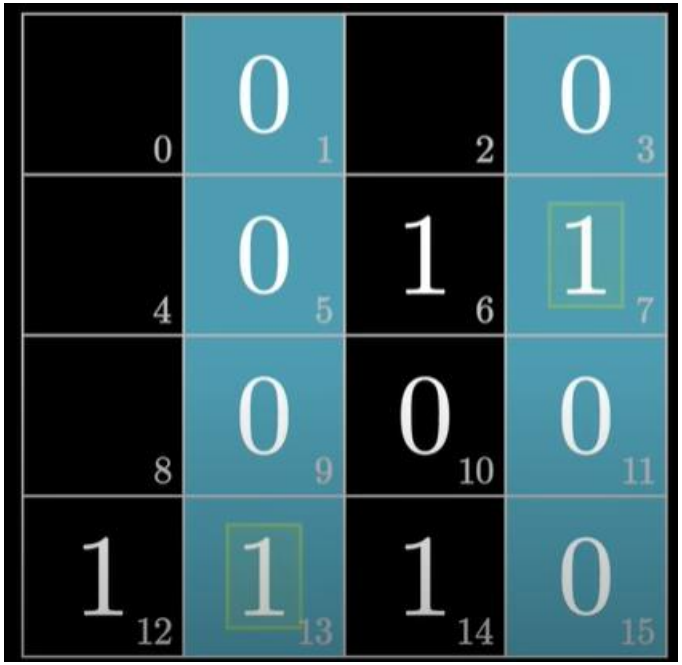


Error-Correcting Codes

Hamming code:

- Find the parity bits p1,p2,p4,p8.
- To find p1 count the number of 1's in odd columns(C1 and C3) . It is 2 (even) so p1=0

C0	C1	C2	C3
0	1	2	0 3
4	0 5	1 6	1 7
8	0 9	0 10	0 11
1 12	1 13	1 14	0 15

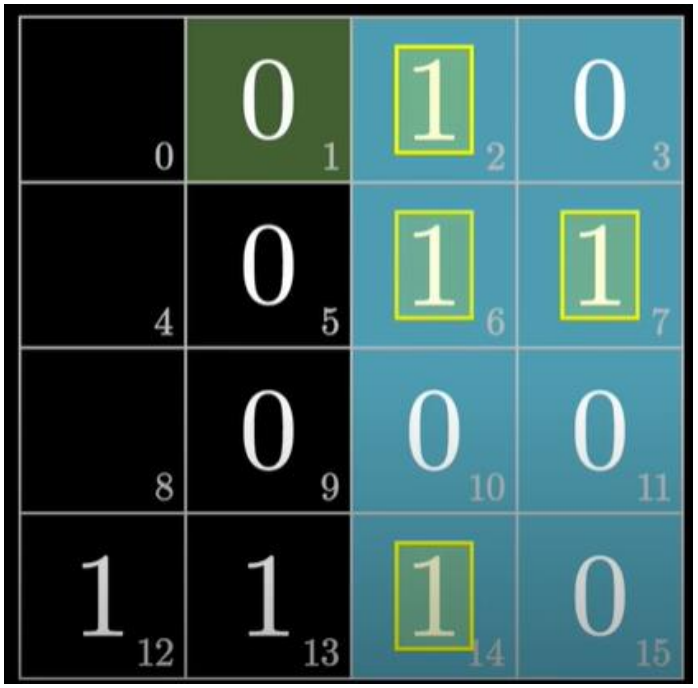


Error-Correcting Codes

Hamming code:

- To find p2 count the number of 1's in C2 and C3. It is 3 (odd) so p2=1

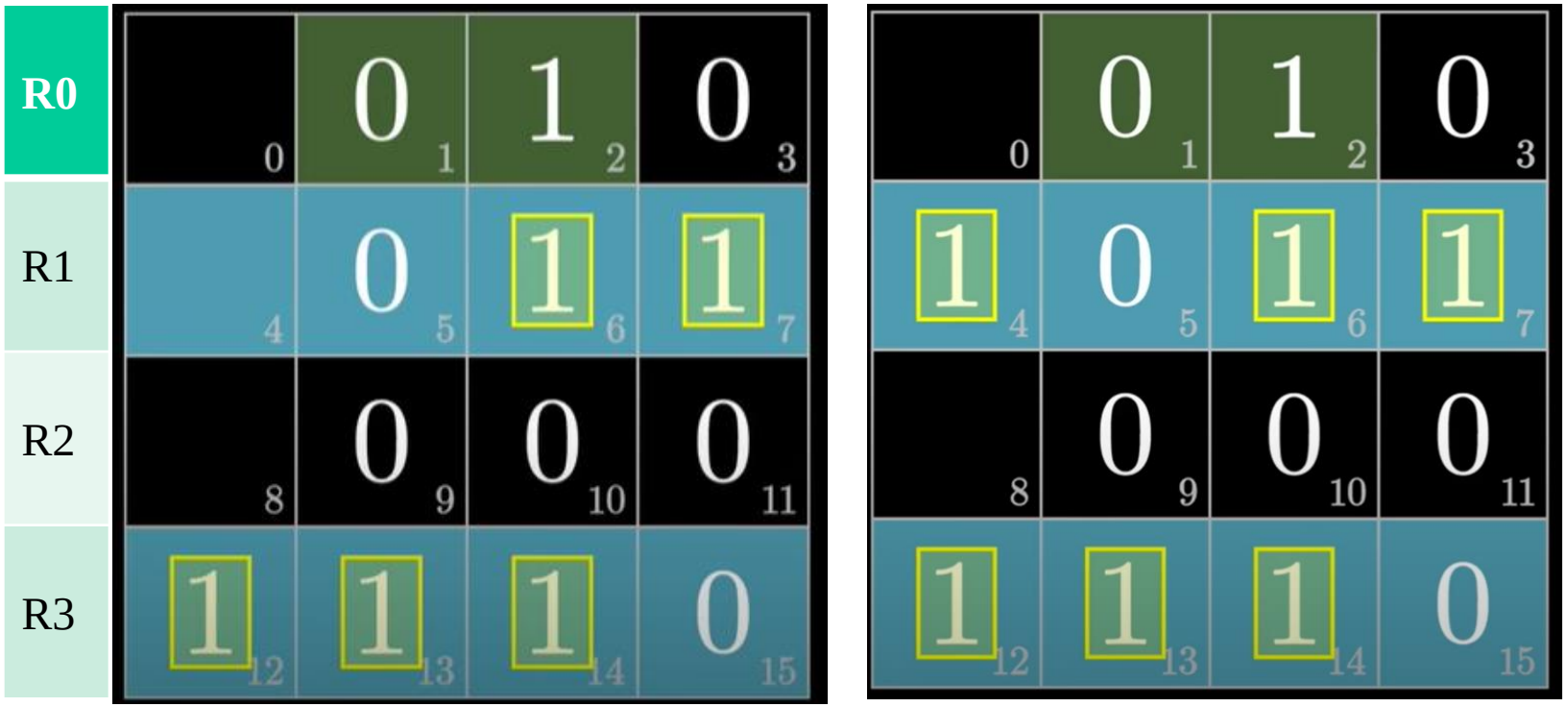
C0	C1	C2	C3
0	0	0	0
4	0	1	1
8	0	0	0
1	1	1	0
12	13	14	15



Error-Correcting Codes

Hamming code:

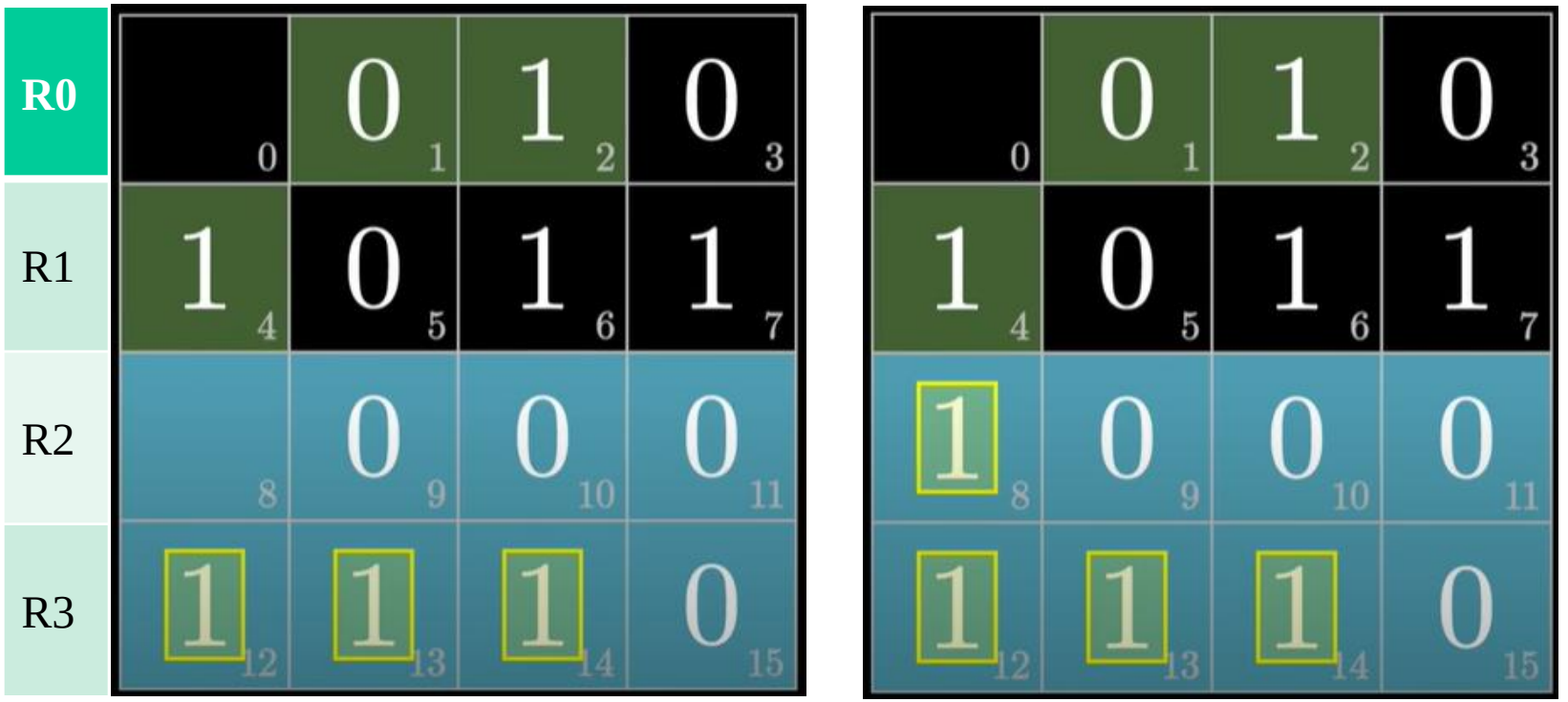
- To find p4 count the number of 1's in R1,R3,R5 and so on. It is 5 (odd) so p4=1



Error-Correcting Codes

Hamming code:

- To find p8 count the number of 1's in R2,R3,R6,R7 and so on. It is 3 (odd) so p8=1



Error-Correcting Codes

Hamming code:

Error-Correcting Codes

Hamming code:

- The receiver will preform the same parity computation over the received block. If all parity bits at the receiver side are 0's then no bits are corrupted as shown below.

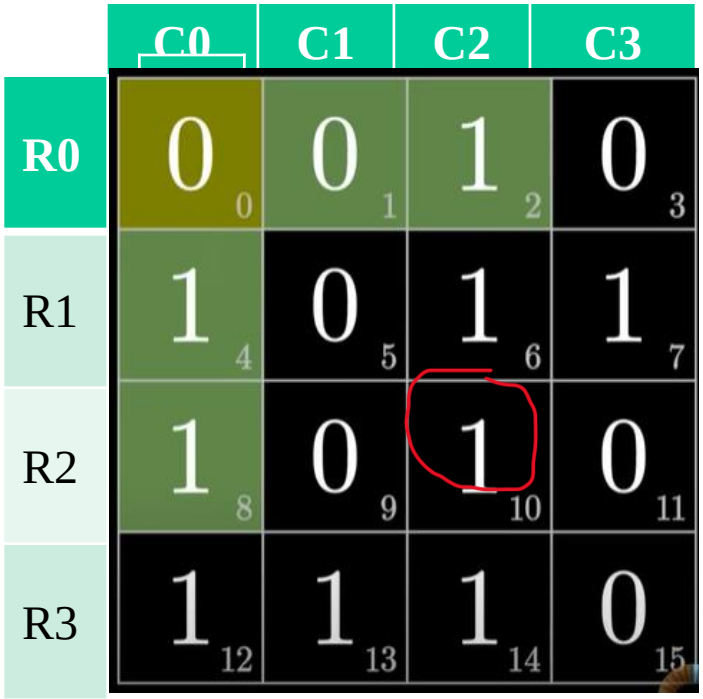
	C0	C1	C2	C3
R0	0	0 1	1 2	0 3
R1	1 4	0 5	1 6	1 7
R2	1 8	0 9	0 10	0 11
R3	1 12	1 13	1 14	0 15

p1 = 0
p2 = 0
p4 = 0
p8 = 0

Error-Correcting Codes

Hamming code:

- Suppose that receiver has received the following block. We can see there an error



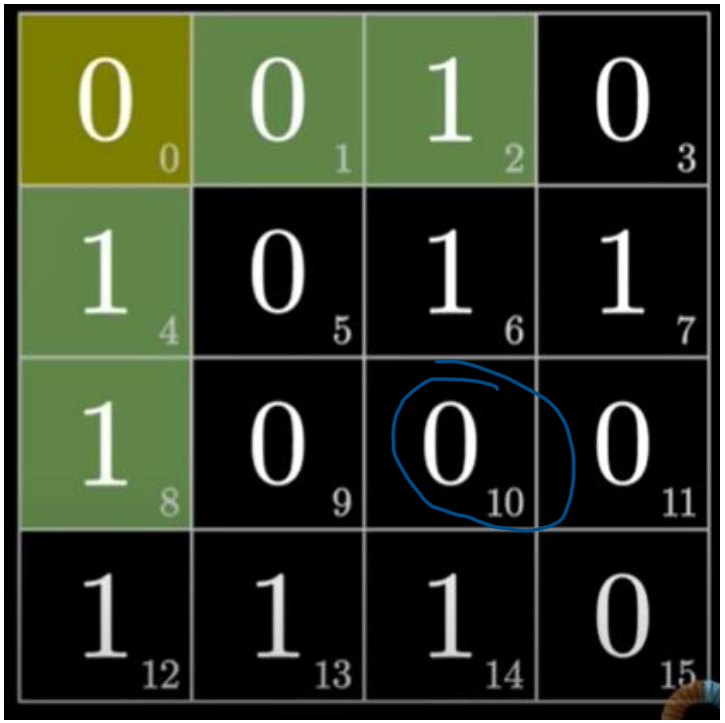
p1 = 0
p2 = 1
p4 = 0
p8 = 1

- The error position is found by adding up bit positions of all parity bits that are 1's. Here p2 and p8 are set. Error pos = 2 + 8 = 10. (10^{th} bit is corrupted.)

Error-Correcting Codes

Hamming code:

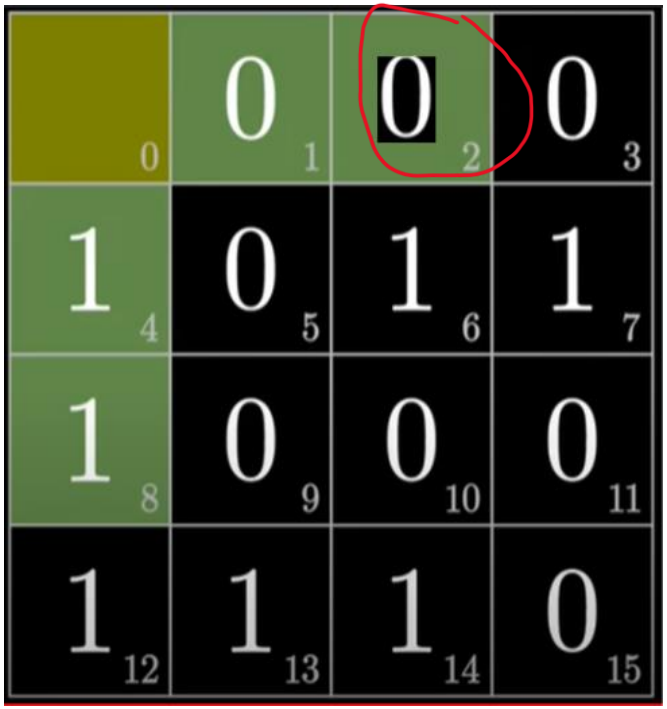
- The receiver will flip the corrupted bit to correct the error.



Error-Correcting Codes

Hamming code:

- If a parity bit is corrupted then only that parity bit will be in problem. No data bits will be affected.



p1 = 0
p2 = 1
p4 = 0
p8 = 0

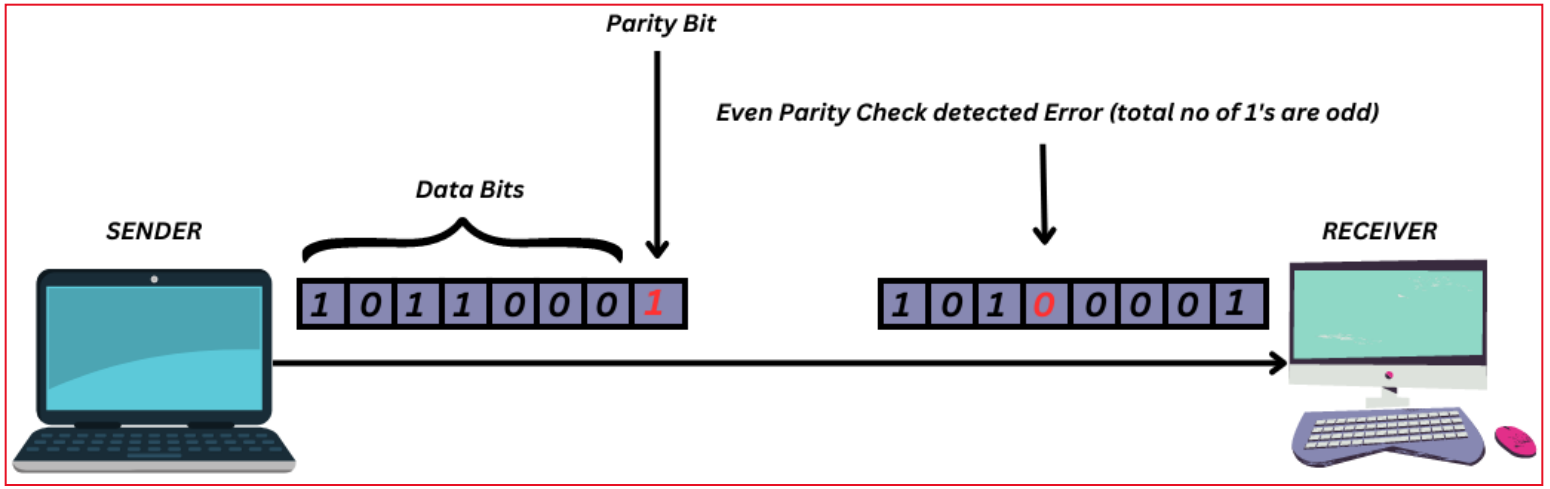
Error-Detecting Codes

- Error-correcting codes are widely used on wireless links, which are notoriously noisy and error prone when compared to copper wire or optical fibers.

- However, over copper wire or fiber, the error rate is much lower, so error detection and retransmission is usually more efficient there for dealing with the occasional error.

Error-Detecting Codes

- **Simple Parity Bit Method :** A parity bit is an extra bit included in binary message to make total number of 1's either odd or even. There are two parity system – even and odd parity checks.



Error-Detecting Codes

Note

- A simple parity-check code is a single-bit error-detecting code in which **n = k + 1 with dmin = 2.**

- A simple parity-check code **can detect an odd number of errors.**

- Receiver requests retransmission if there is an error.

Error-Detecting Codes

Two-dimensional parity-bit Method

- Here dataword to be sent is organized as matrix
- A parity bit is computed separately for each row and column of the matrix
- Each row is a block. Thus, we send one extra block and one extra bit for each block.
- The whole matrix is then sent to the receiver, which finds parity bits for each row and column.

Error-Detecting Codes

Two-dimensional parity-bit Method

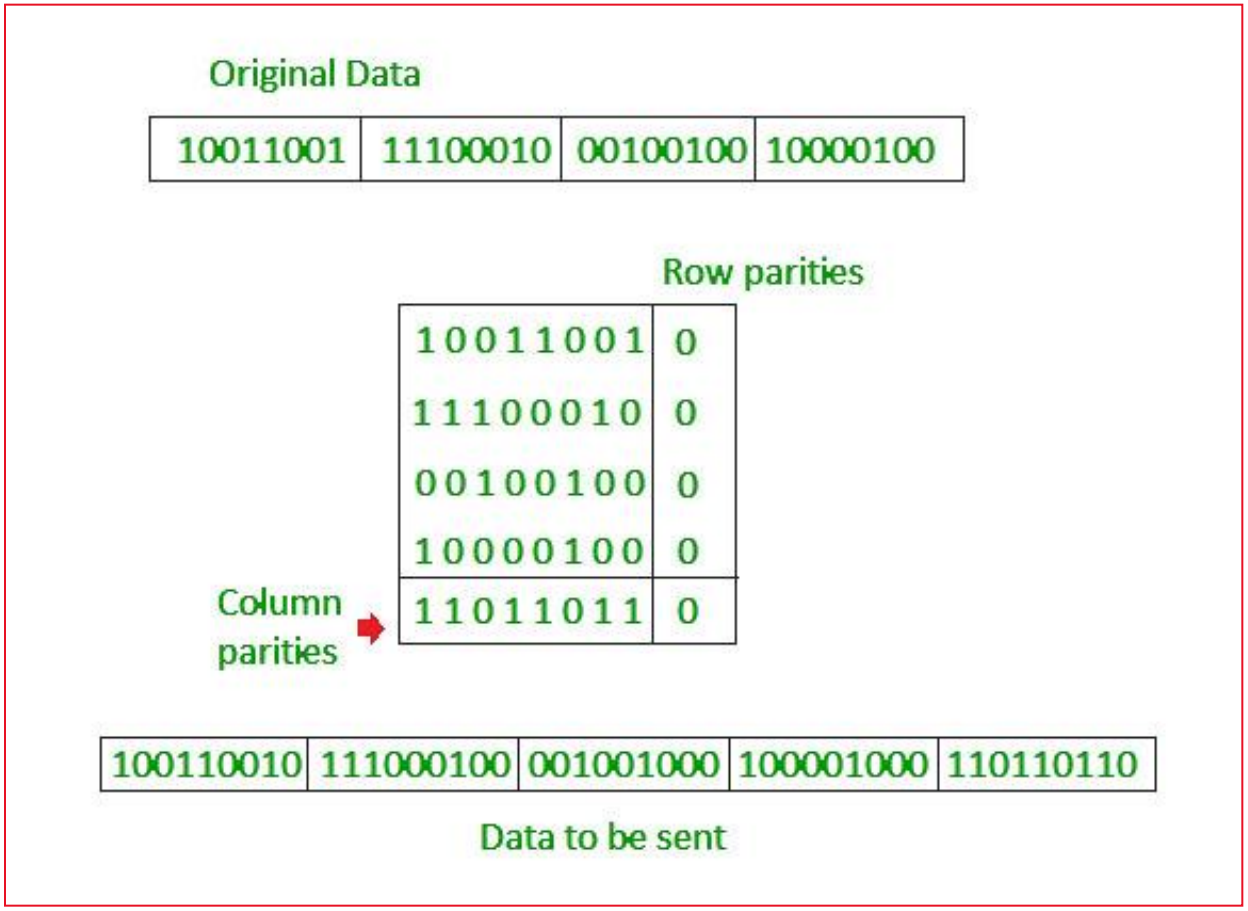
- The receiver finds the parity bit for each row and column

- If any one of them is wrong, the receiver requests a retransmission of the data.

Error-Detecting Codes

Two-dimensional parity-bit Method

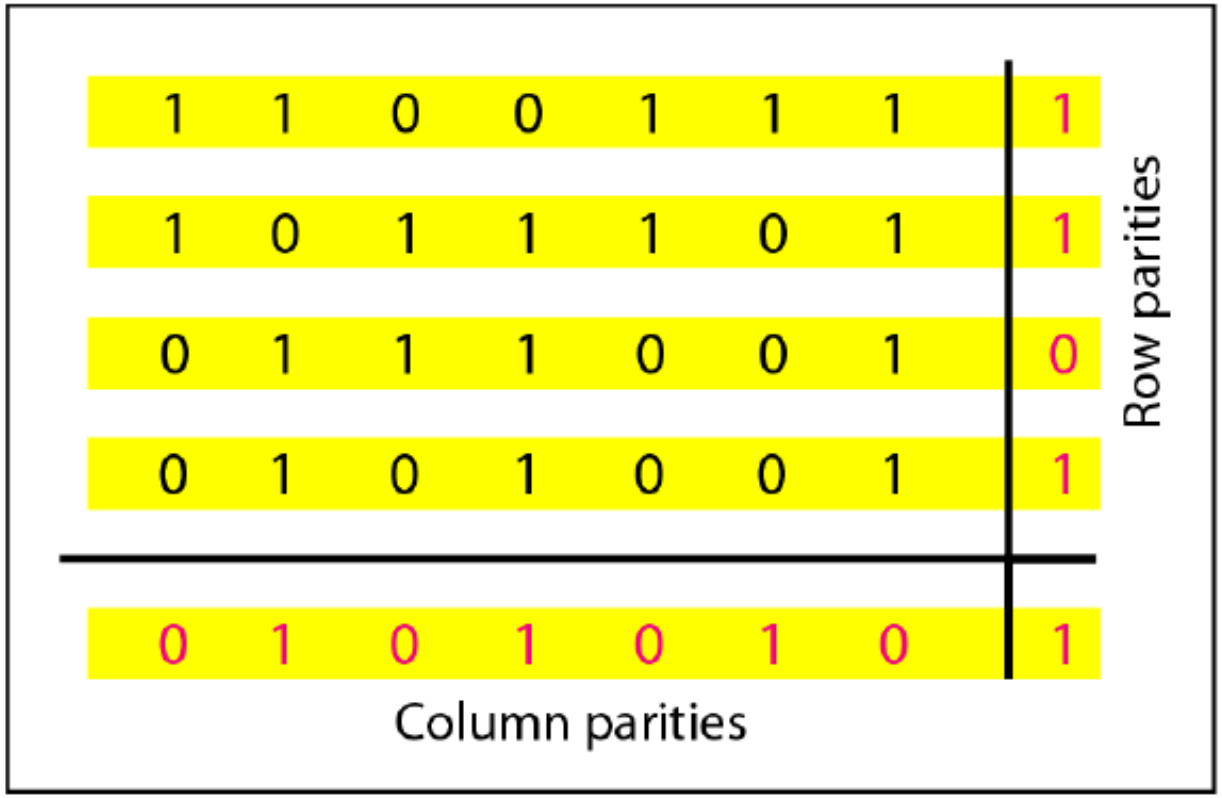
- **working :**



Error-Detecting Codes

Two-dimensional parity-bit Method

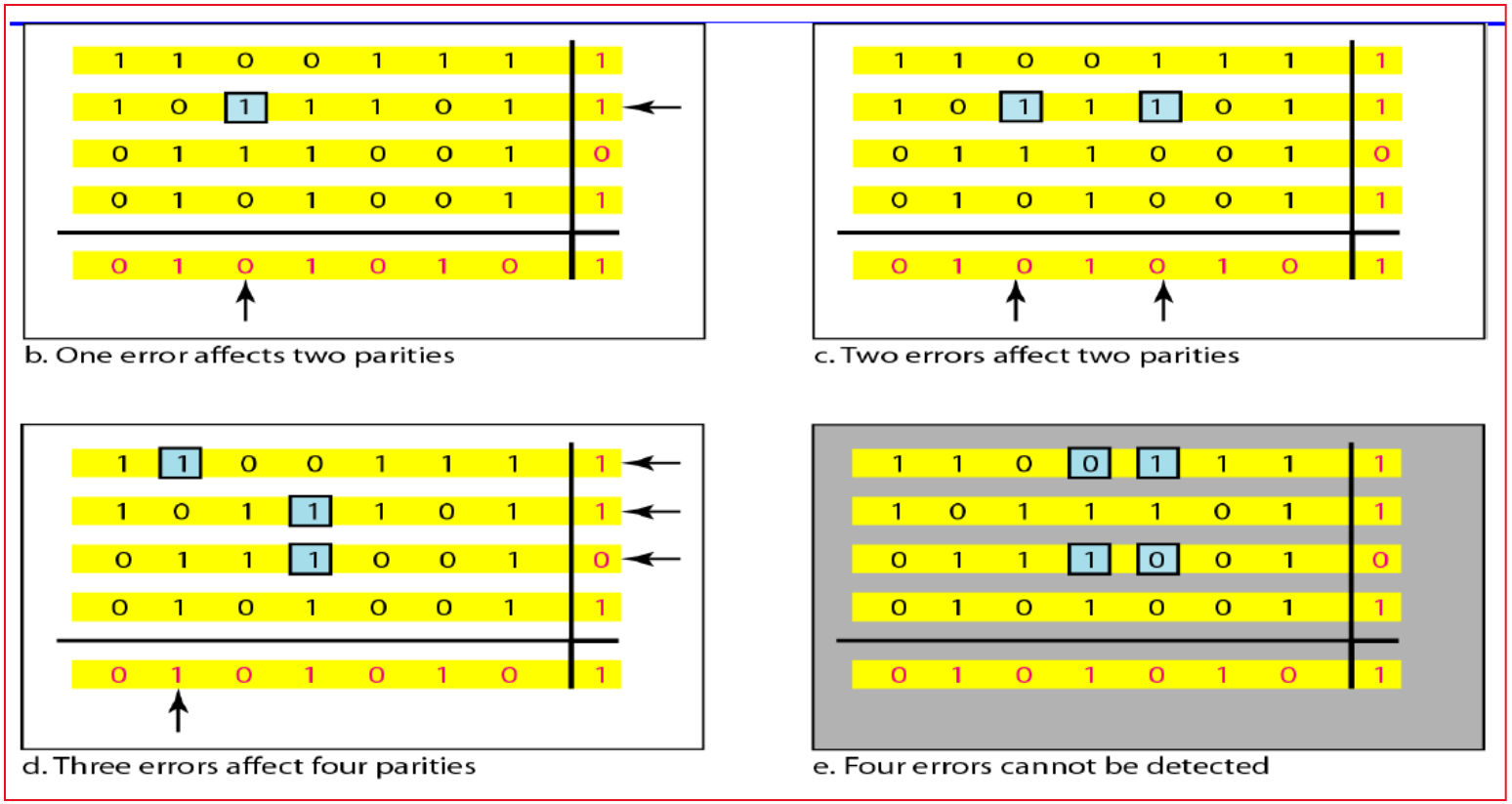
- **Example :**



Error-Detecting Codes

Two-dimensional parity-bit Method

- **Example :**



Error-Detecting Codes

Two-dimensional parity-bit Method

- It clear that Two-dimensional parity-bit can detect upto 3 bit errors.
- Used for sending large blocks data.
- Efficient when compared with single-bit parity bit method

Error-Detecting Codes

CRC (Cyclic Redundancy Check) or or Polynomial codes :

- Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only.

- A k-bit frame is regarded as the coefficient list for a polynomial with k terms, ranging from $x^{k-1}$to x^0.

- Example : 110001 has its polynomial : $\boldsymbol{x^5 + x^4 + x^0}$

Error-Detecting Codes

CRC (Cyclic Redundancy Check) or Polynomial codes :

- Polynomial arithmetic is done modulo 2, in which addition and subtraction are identical to exclusive OR.
- The sender and receiver must agree upon a generator polynomial, G(x), in advance.

- Both the high- and low-order bits of the generator must be 1.

Error-Detecting Codes

CRC (Cyclic Redundancy Check) or Polynomial codes :

- **The algorithm for computing the checksum is as follows:**

1. Let r be the degree of G(x). Append r zero bits to the dataword so it now contains m + r bits and corresponds to the polynomial $x^r M(x)$.
2. Divide the dataword corresponding to $x^r M(x)$ by bit string corresponding to G(x) using modulo 2 division.
3. Replace the lower r bits in the dataword with r bit remainder (**check bits**) to get the codeword for transmission.

Error-Detecting Codes

CRC (Cyclic Redundancy Check) or Polynomial codes :

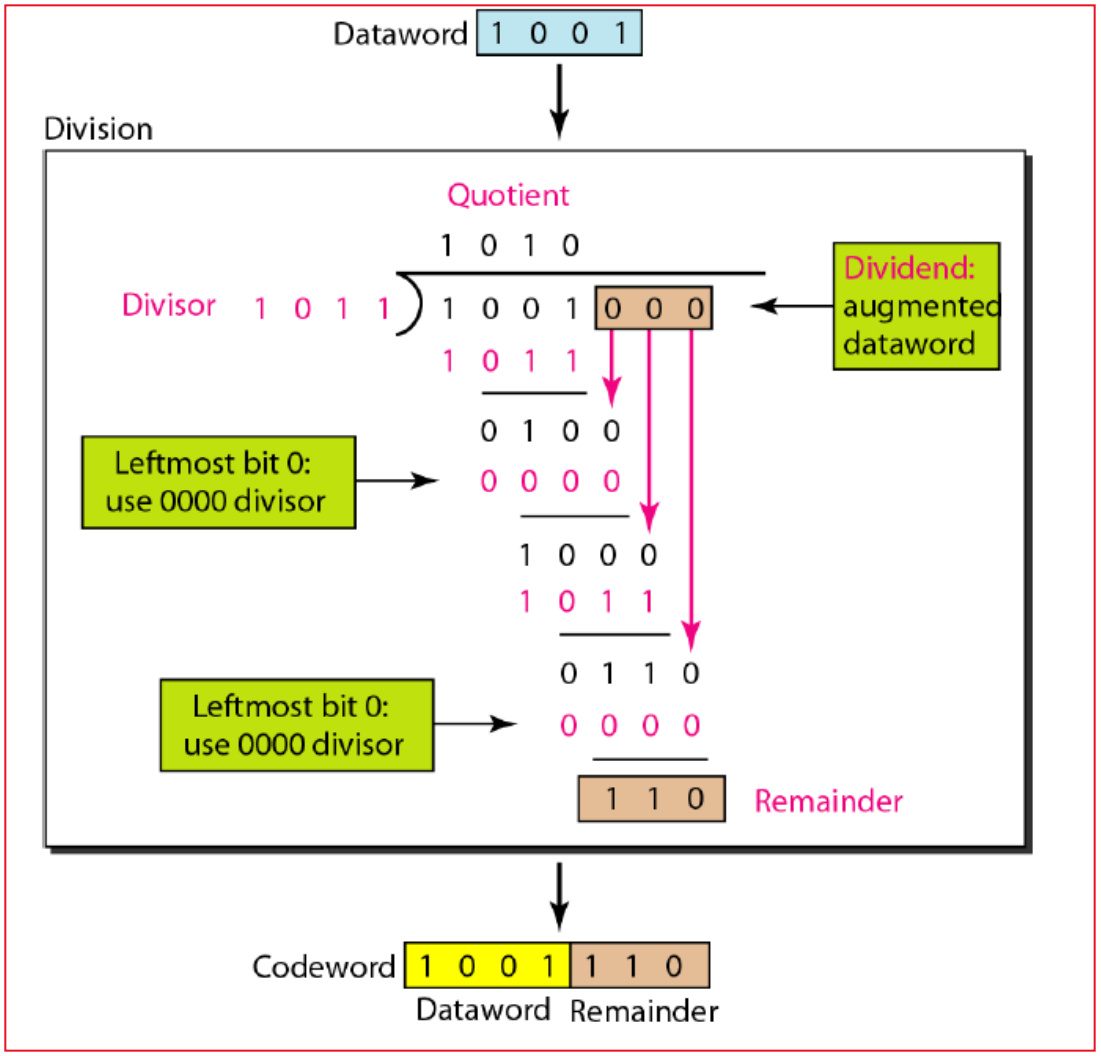
- **The algorithm for computing the checksum is as follows:**

1. The receiver will carry out the same operations and if there is a non-zero reminder, codeword received has an error.

2. If codeword received is zero, codeword received may or may not have an error.

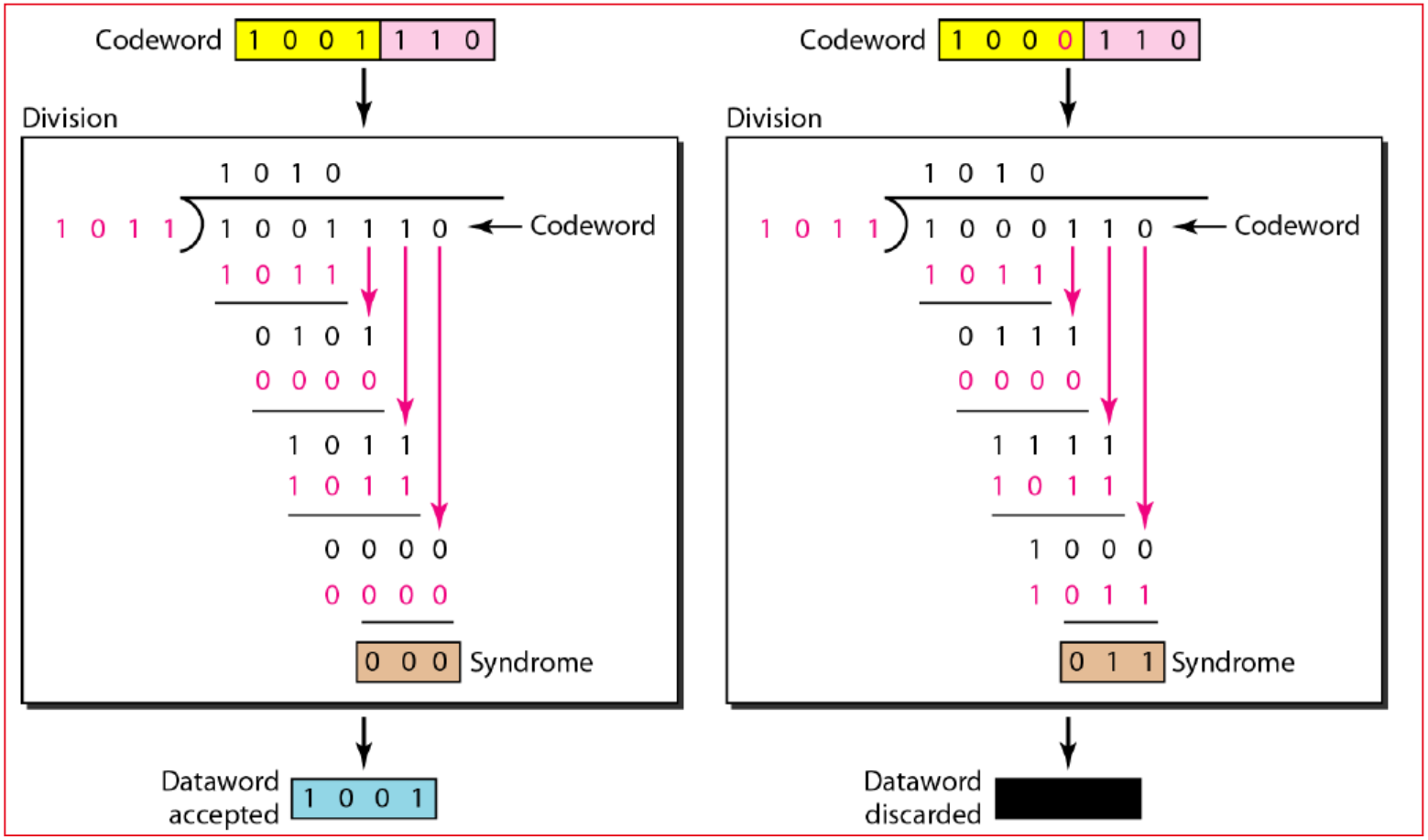
Error-Detecting Codes

CRC (Cyclic Redundancy Check) or Polynomial codes :



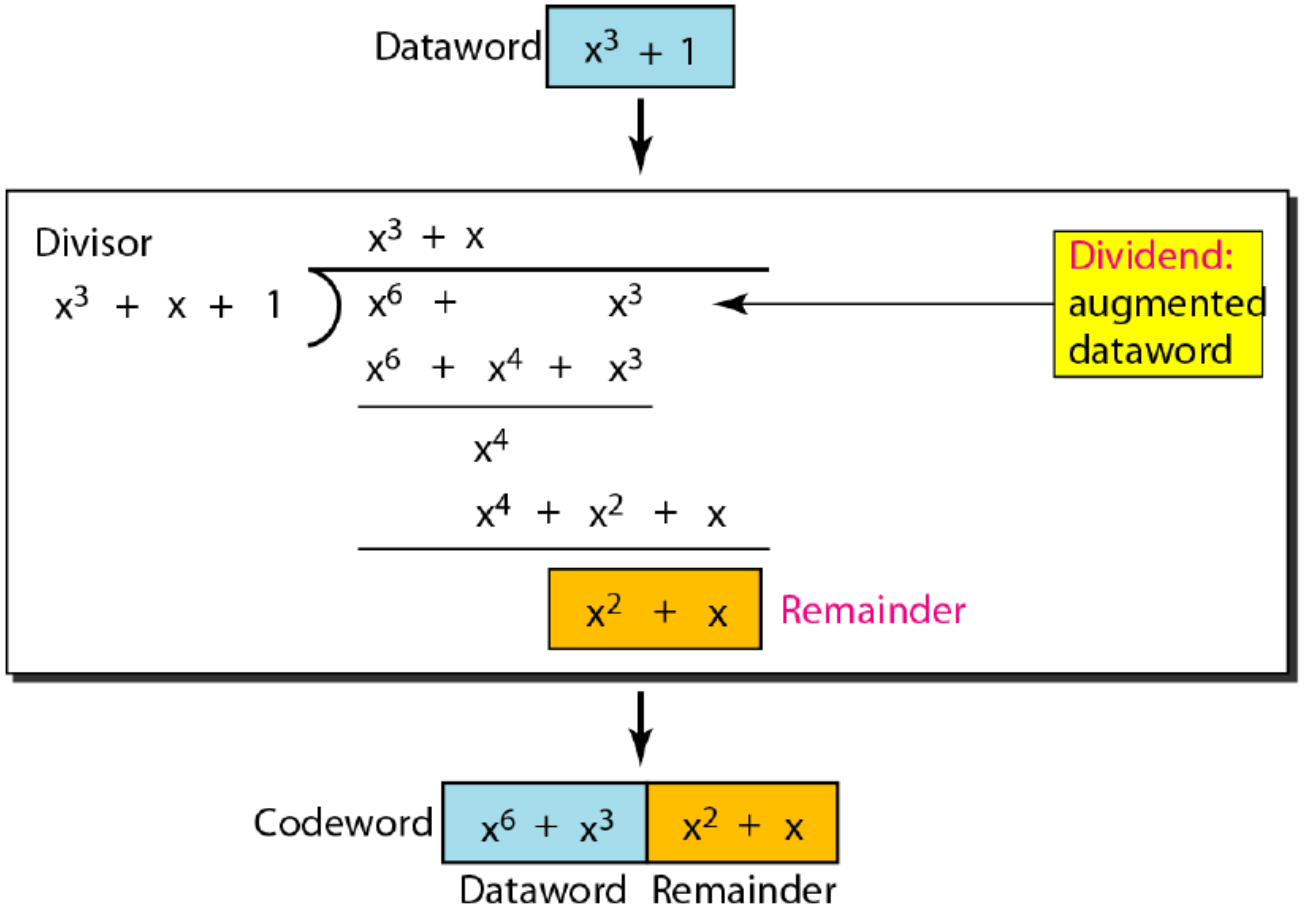
Error-Detecting Codes

CRC (Cyclic Redundancy Check) or Polynomial codes :



Error-Detecting Codes

CRC (Cyclic Redundancy Check) or Polynomial codes :



Elementary Data Link Protocols

1. An Unrestricted Simplex Protocol
2. A Simplex Stop-and-Wait Protocol
3. A Simplex Protocol for a Noisy Channel

Elementary Data Link Protocols

- When the data link layer accepts a packet, it encapsulates the packet in a frame by adding a **data link header and trailer to it.**

- Thus, a frame consists of an embedded packet, some control information (in the header), and a checksum (in the trailer). The frame is then transmitted to the data link layer on the other machine.

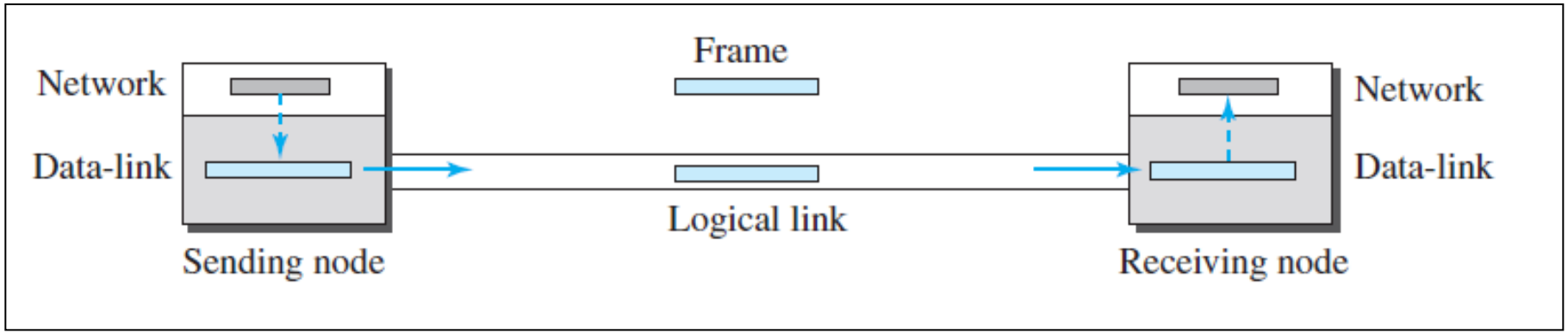
- The checksum (in the trailer) is computed by the hardware at the sender.

Elementary Data Link Protocols

- When a frame arrives at the receiver, the hardware computes the checksum.
- If the checksum is incorrect, the data link layer is so informed. If the inbound frame arrived undamaged, the data link layer is also informed so that it can acquire the frame for inspection using from physical layer.
- As soon as the receiving data link layer has acquired an undamaged frame, it checks the control information in the header, and if everything is all right, passes the packet portion to the network layer.

An Unrestricted Simplex Protocol

- Our first protocol is a simple protocol with neither flow nor error control.
- That is Channel is error free and the receiver can never be overwhelmed with incoming frames
- We assume that the receiver can immediately handle any frame it receives.

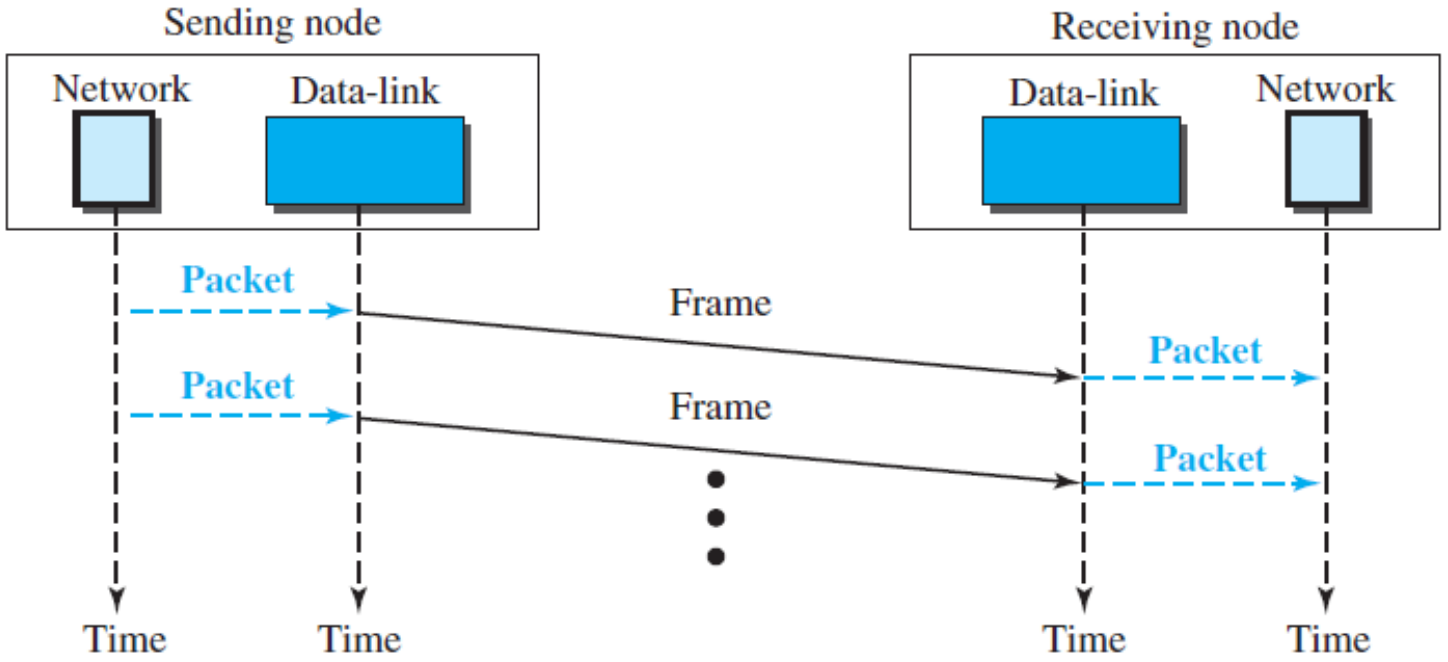


An Unrestricted Simplex Protocol

- The data-link layer at the sender gets a packet from its network layer, makes a frame out of it, and sends the frame.
- The data-link layer at the receiver receives a frame from the link, extracts the packet from the frame, and delivers the packet to its network layer.
- The data-link layers of the sender and receiver provide transmission services for their network layers.

An Unrestricted Simplex Protocol

- Figure shows an example of communication using this protocol. It is very simple. The sender sends frames one after another without even thinking about the receiver



A Simplex Stop-and-Wait Protocol

- The communication channel is still assumed to be error free however, and the data traffic is still simplex.
- However, we assume that presence of flow control. (receiving data link layer has a finite amount of buffer space to the frames)
- The sender starts out by fetching a packet from the network layer, using it to construct a frame, and sending it on its way.

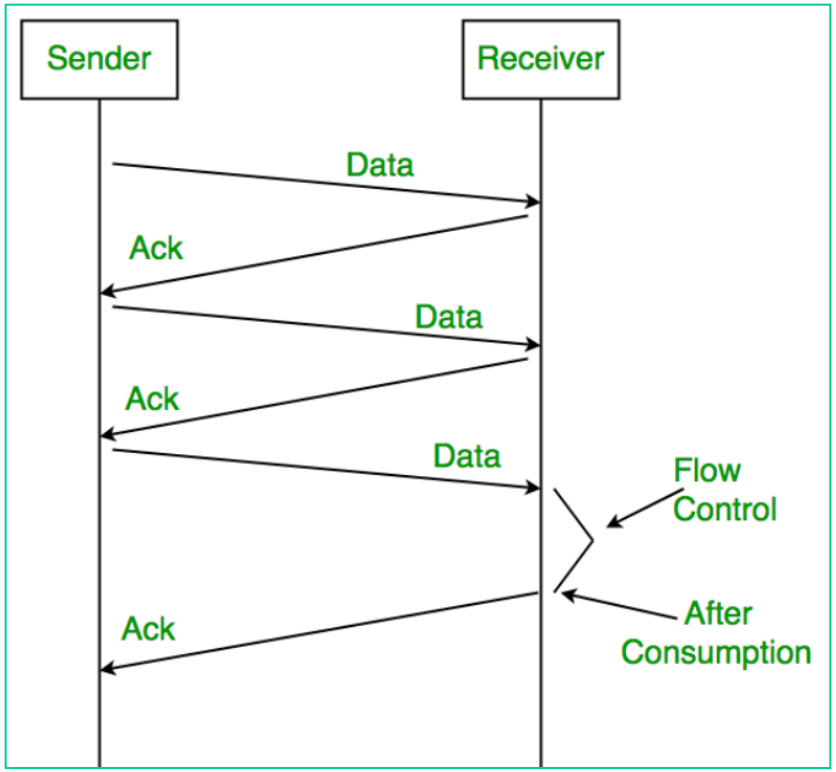
A Simplex Stop-and-Wait Protocol

- The communication channel is still assumed to be error free however, and the data traffic is still simplex.
- The sender starts out by fetching a packet from the network layer, using it to construct a frame, and sending it on its way.
- The sender must wait until an acknowledgement frame arrives before looping back and fetching the next packet from the network layer.
- The sending data link layer need not even inspect the incoming frame: there is only one possibility. The incoming frame is always an acknowledgement.

A Simplex Stop-and-Wait Protocol

- Although data traffic in this example is simplex, data flows both directions at a time.
- That is , first the sender sends a frame, then the receiver sends a frame (ack), then the sender sends another frame, then the receiver sends another one(ack), and so on.

- A **half- duplex physical channel** would suffice here.



A Simplex Protocol for a Noisy Channel

- We assume Frames may be either damaged or lost completely.

- However, we assume that if a frame is damaged in transit, the receiver hardware will detect this when it computes the checksum.

- If the frame is damaged in such a way that the checksum is nevertheless correct, an unlikely occurrence, this protocol can fail (i.e., deliver an incorrect packet to the network layer).

A Simplex Protocol for a Noisy Channel

Working :

- After transmitting a frame and starting the timer, the sender waits for something exciting to happen. Only three possibilities exist: **an acknowledgement frame arrives undamaged**, **a damaged acknowledgement frame staggers in**, or **the timer expires**.

- If a valid acknowledgement comes in, the sender fetches the next packet from its network layer and repeats the process.

A Simplex Protocol for a Noisy Channel

Working :

- When a valid frame arrives at the receiver, its sequence number is checked to see if it is a duplicate. If not, it is accepted, passed to the network layer, and an acknowledgement is generated.

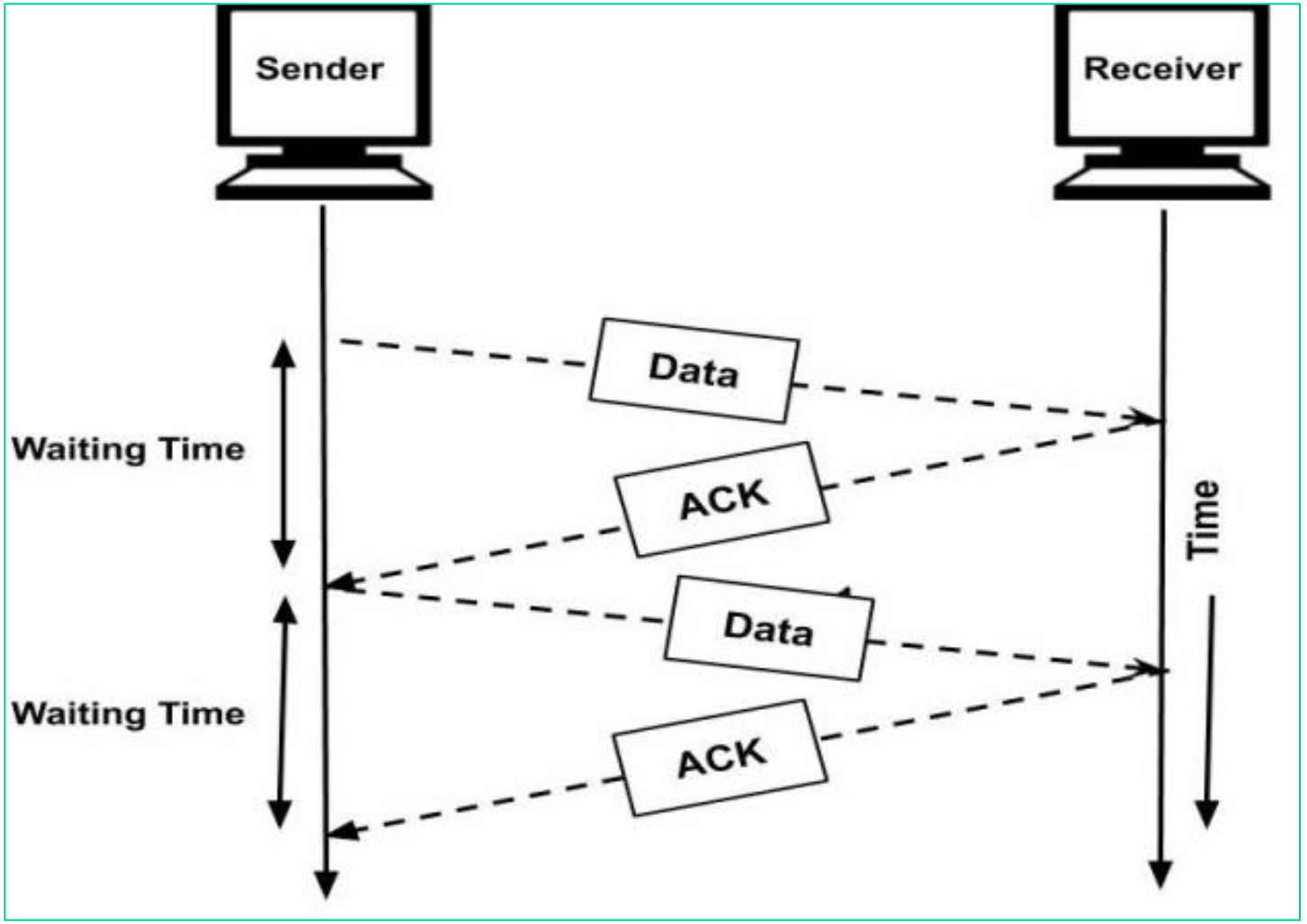
- Note that each frame sent by the sender contains a unique sequence number. This helps the receiver to detect and discard any duplicate packets.

.

A Simplex Protocol for a Noisy Channel

Working :

.



A Simplex Protocol for a Noisy Channel

Working :

.

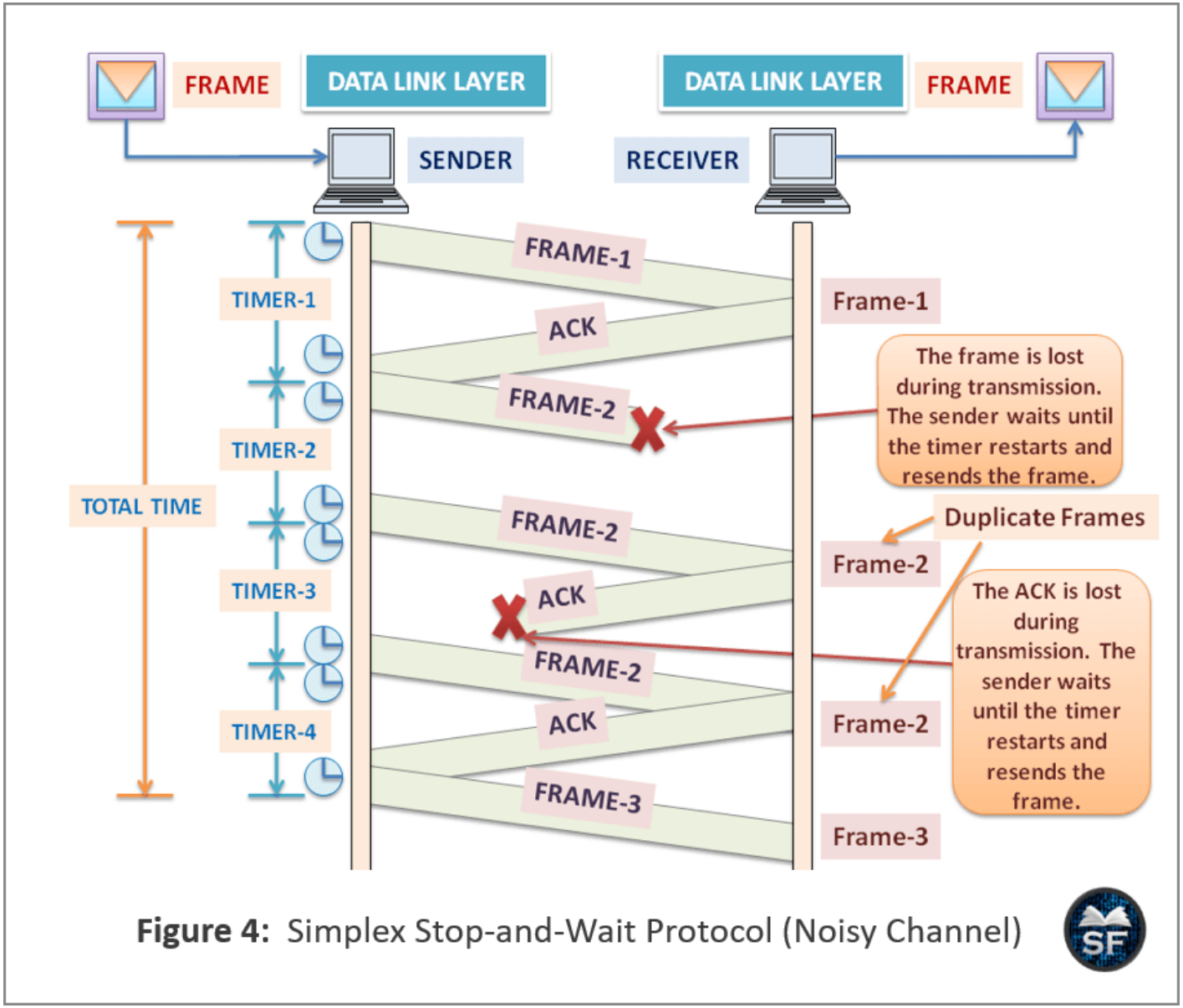


Figure 4: Simplex Stop-and-Wait Protocol (Noisy Channel)

Sliding Window Protocols

Sliding Window Protocols

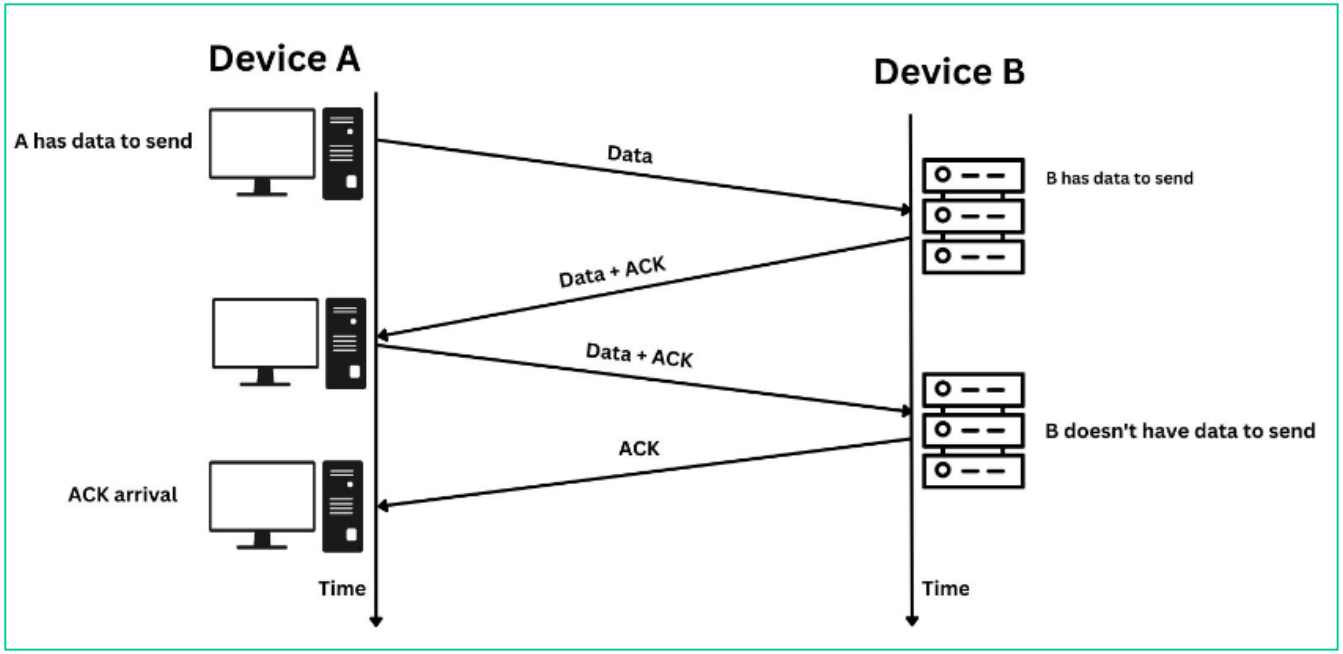
- **A One-Bit Sliding Window Protocol**
- **A Protocol Using Go Back N**
- **A Protocol Using Selective Repeat**

.

Sliding Window Protocols

What is piggybacking ?

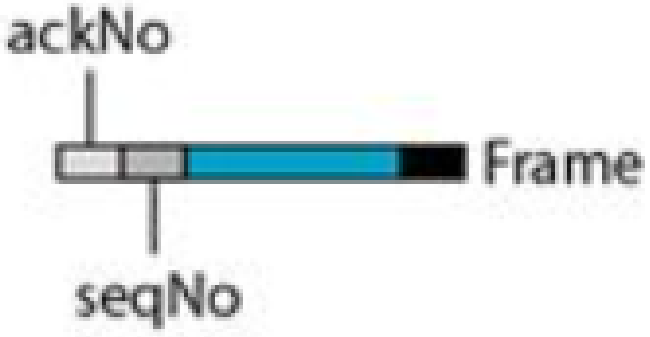
- Instead of immediately sending a separate ack frame, the receiver restrains itself and waits until the network layer passes the next packet. The ack is attached to the outgoing data frame. In effect, the acknowledgement gets a free ride on the next outgoing data frame from the receiver. This is known as **piggybacking**



Sliding Window Protocols

What is piggybacking ?

- The principal advantage of using piggybacking over having distinct acknowledgement frames is a better use of the available channel bandwidth.
- A separate ack frame would need **a header, the acknowledgement, and a checksum,** where as the ack field in the frame header costs only a few bits



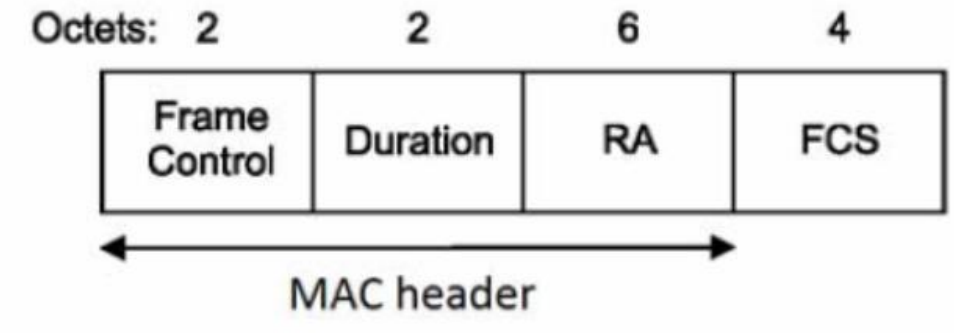
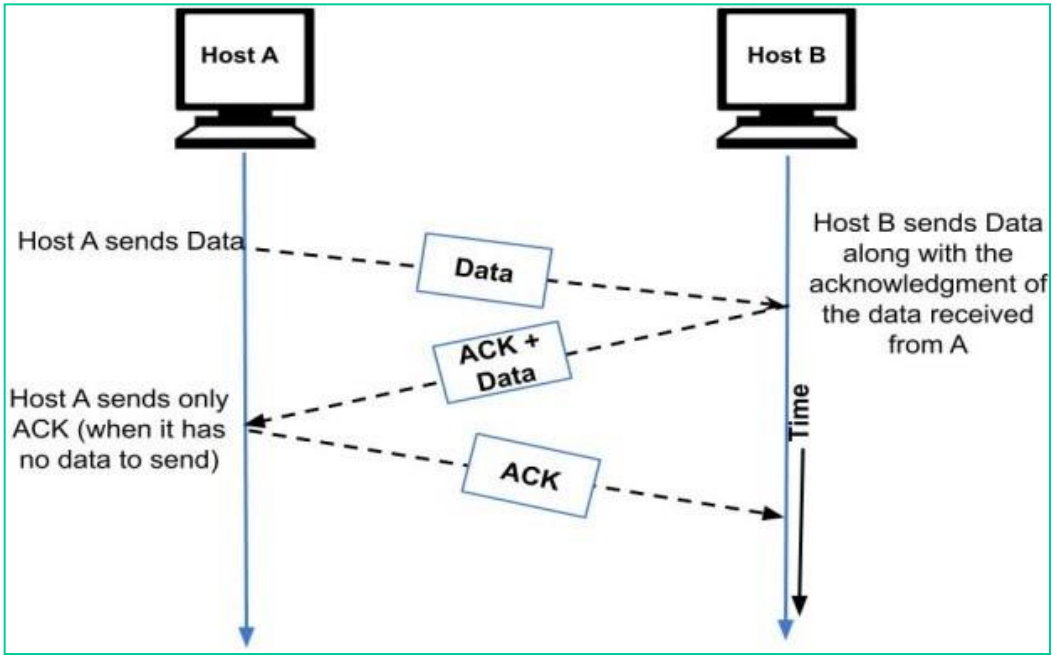


Figure 9-22—Ack frame

Sliding Window Protocols

What is piggybacking ?

- Disadvantage of **piggybacking** is that If the data link layer waits longer than the sender's timeout period, sender's the frame will be retransmitted.
- The solution is to send a **separate acknowledgement** frame if no new packet has arrived by the end of waiting time period.



Sliding Window Protocols

- For bidirectional communication DL uses **Sliding Window Protocols**
- In all sliding window protocols at any instant of time, the sender maintains a set of sequence numbers corresponding to frames it is permitted to send.
- These frames are said to fall within **the sending window.**
- Similarly, the receiver also maintains **a receiving window** corresponding to the set of frames it is permitted to accept.

Sliding Window Protocols

sliding window protocols

- The sender's window and the receiver's window need not have the same lower and upper limits or even have the same size.

- In some protocols they are fixed in size, but in others they can grow or shrink over the course of time as frames are sent and received.

Sliding Window Protocols

sliding window protocols

- The sequence numbers within the sender's window represent frames that have been sent or can be sent but are as yet not acknowledged.

- Whenever a new packet arrives from the network layer, it is given the next **highest sequence number**, and the upper edge of the window is advanced by one.

- When an acknowledgement comes in, the lower edge is advanced by one. In this way the window continuously maintains a list of unacknowledged frames.

Sliding Window Protocols

sliding window protocols

- Setting the smallest range for sequence numbers minimizes the frame size.

- If the sequence number is of m bits, then the sequence number is restricted to 0 to $2^m - 1$.

- For example, sequence number is of 3 bits, then the sequence number is range restricted to 0 to 7. These number will be repeated for next 8 frames and so on.

  (based on modulo-2 arithmetic).

Sliding Window Protocols

- The receiving DL’s window corresponds to the frames it may accept.
- Any frame falling outside the window is discarded without comment.
- When a frame whose sequence number is equal to the lower edge of the window is received, it is passed to the network layer, an acknowledgement is generated, and the window is rotated by one.
- Unlike the sender's window, the receiver's window always remains at its initial size

Sliding Window Protocols

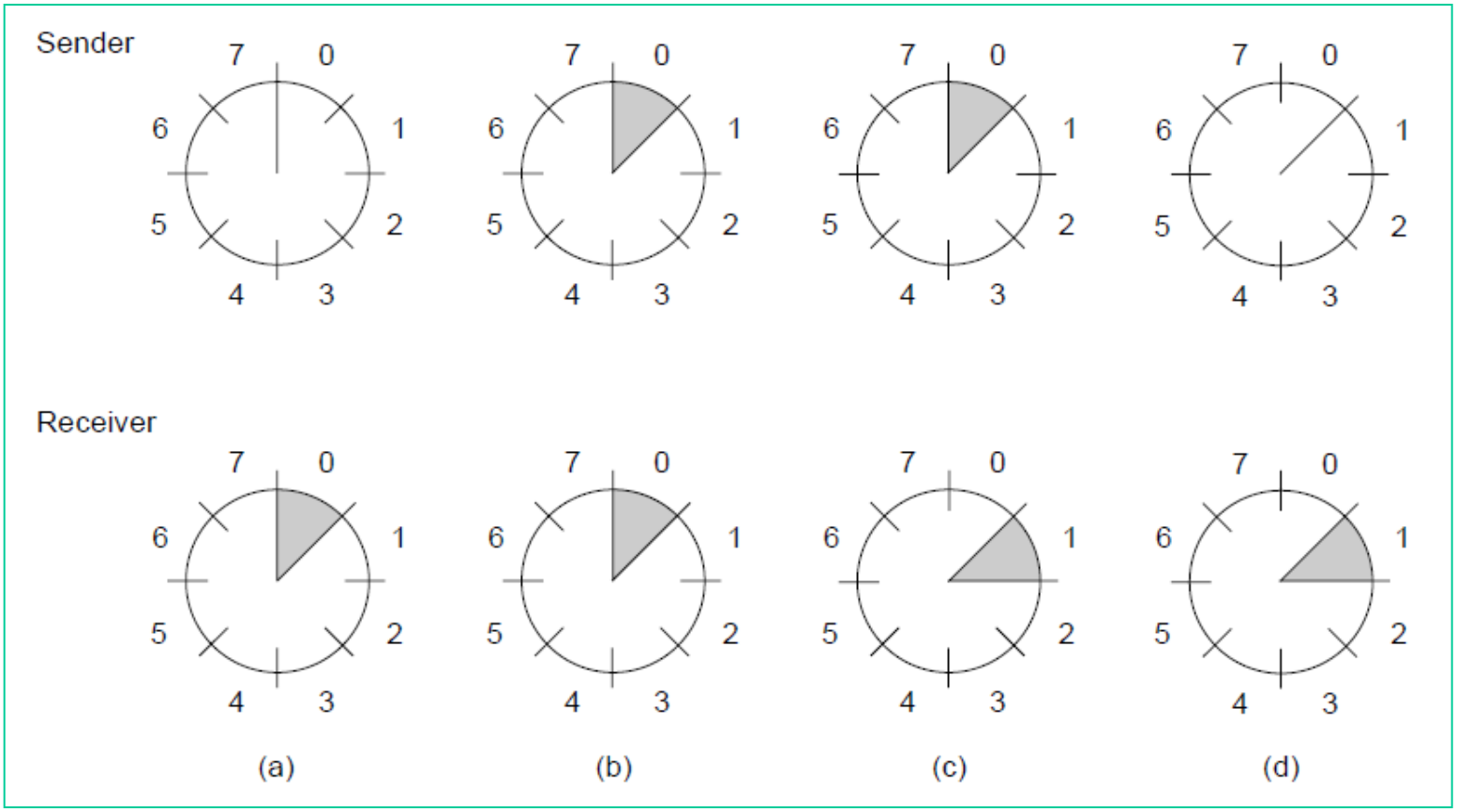
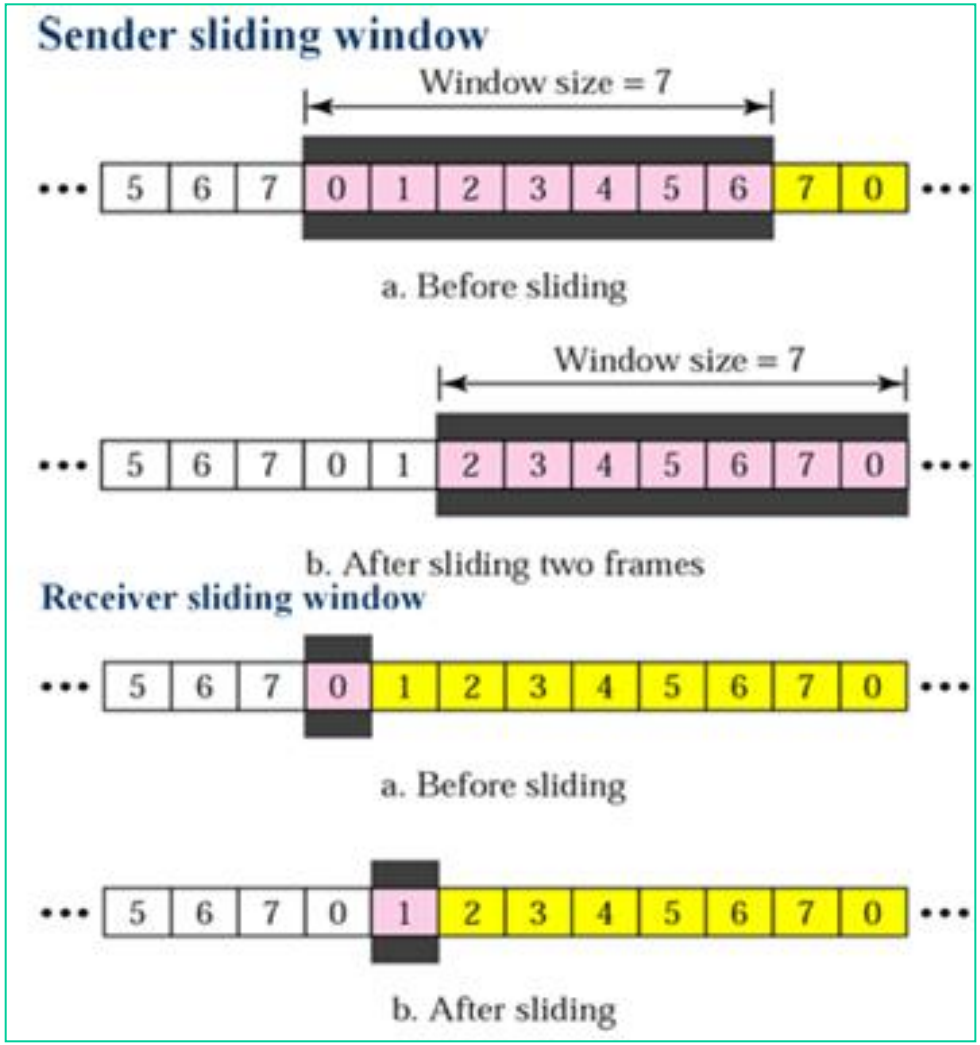


Fig. 3-13. A sliding window of size 1, with a 3-bit sequence number. (a) Initially. (b) After the first frame has been sent. (c) After the first frame has been received. (d) After the first acknowledgement has been received.

Sliding Window Protocols



Sliding Window Protocols

- Since frames currently within the sender's window may ultimately be lost or damaged in transit, the sender must keep all these frames in its memory for possible retransmission.

- Thus, if the maximum window size is **n**, the sender needs n buffers to hold the unacknowledged frames.

- If the window ever grows to its maximum size, the sending data link layer must forcibly shut off the network layer until another buffer becomes free.

A One-Bit Sliding Window Protocol

- In one – bit sliding window protocol, the size of the window is 1. Thus sender/receiver is allowed send one frame at a time (not multiple frames)

- This protocol provides for **full – duplex communications**. Hence, the acknowledgment is attached along with the next data frame to be sent by **piggybacking.**

- So the sender/receiver transmits a frame, waits for its acknowledgment, then transmits the next frame. Thus, it uses the concept of **stop and waits** for the protocol.

A One-Bit Sliding Window Protocol

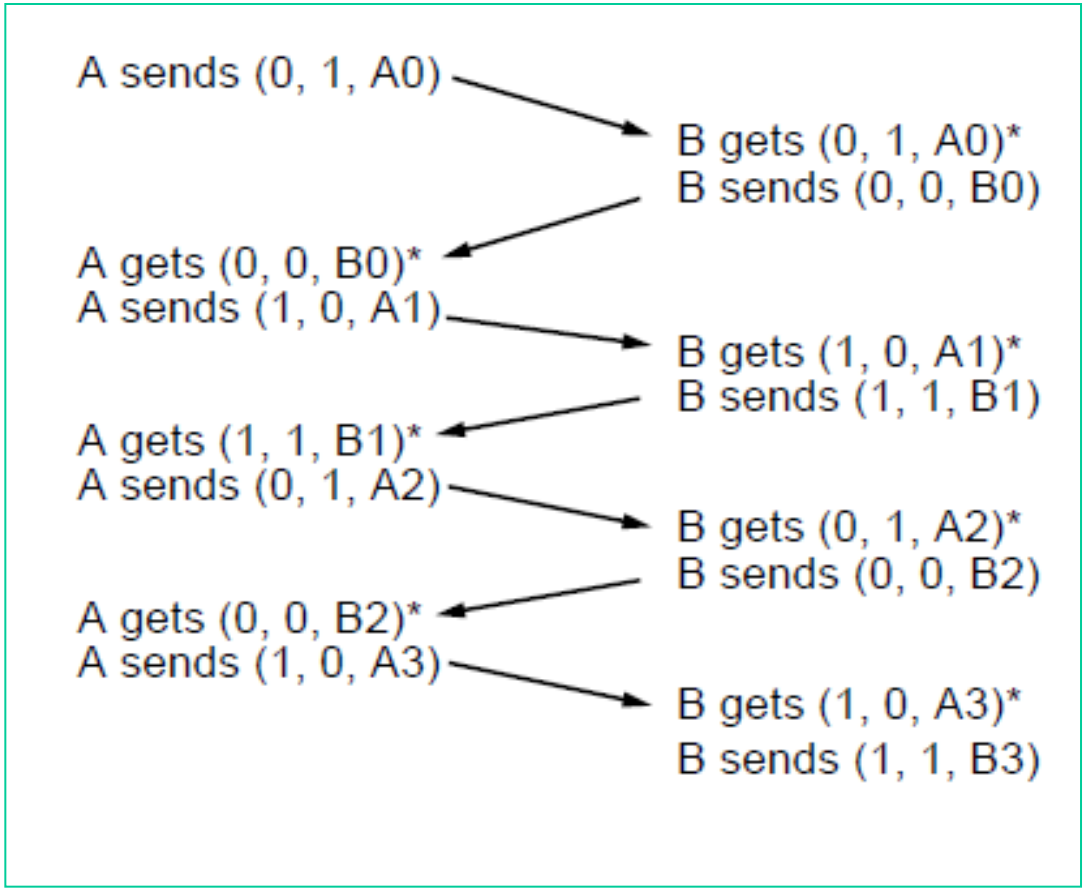
- Each frame sent has three fields : **sequence number, ack number and frame number.**
- Sequence number is the Sequence number of next frame to be sent.
- The ack field contains the **sequence number of the last frame received without error**.

A One-Bit Sliding Window Protocol

- This protocol uses 1 bit sequence number so sequence number are restricted to 0 and 1.
- This means ack numbers will be restricted to 0 and 1
- Since this is a bi-directional protocol, the same algorithm applies to both the communicating parties.

A One-Bit Sliding Window Protocol

- Normal case. An asterisk indicates where a network layer accepts a packet. The notation is (seq, ack, packet number).

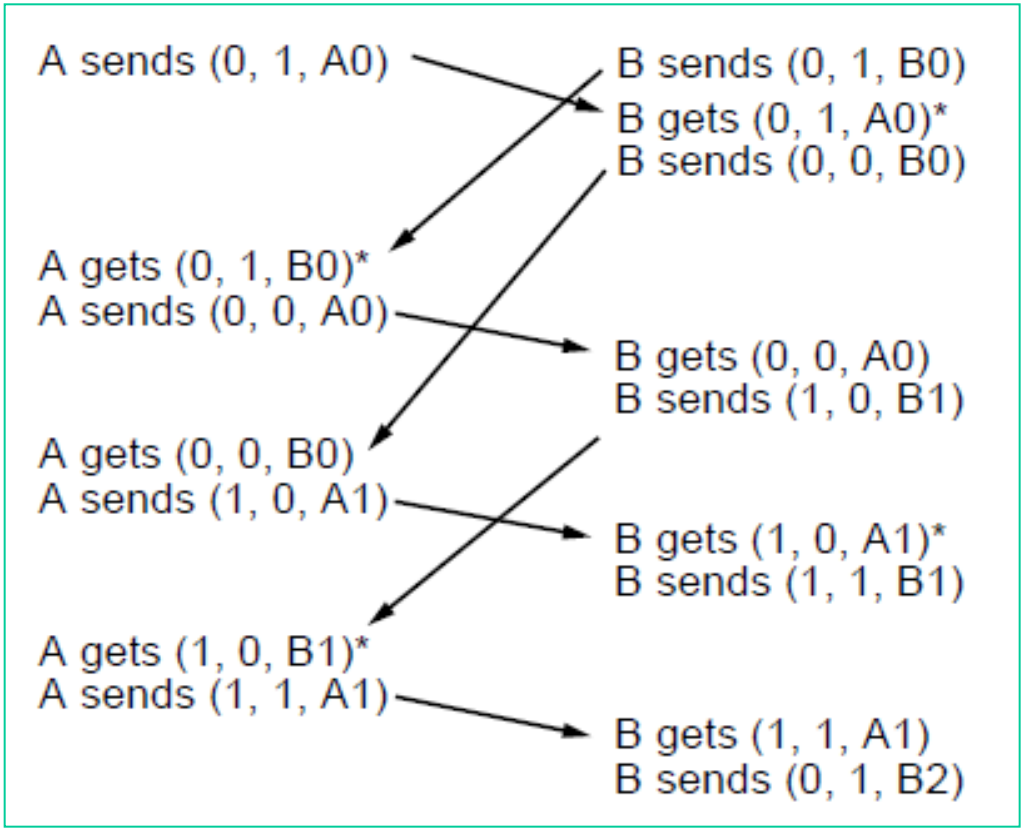


A One-Bit Sliding Window Protocol

- However, difficulty arises when both sides simultaneously send an initial packet as it shown in the next slide.
- Because of this sender and receive will find difficult in synchronization
- This synchronization difficulty is illustrated next

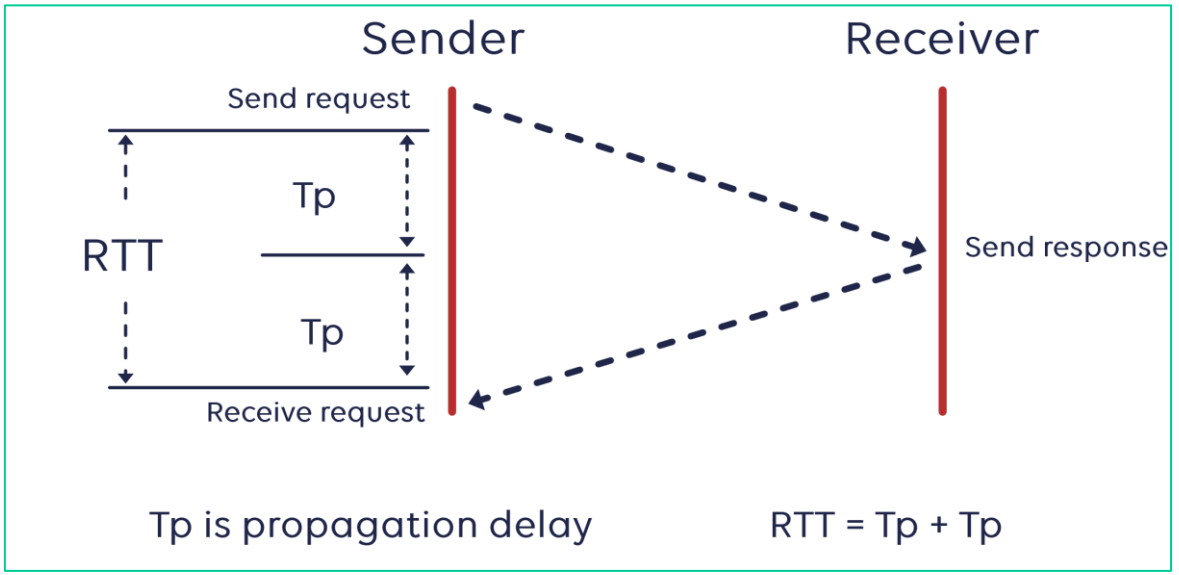
A One-Bit Sliding Window Protocol

- Abnormal case. An asterisk indicates where a network layer accepts a packet. The notation is (seq, ack, packet number).



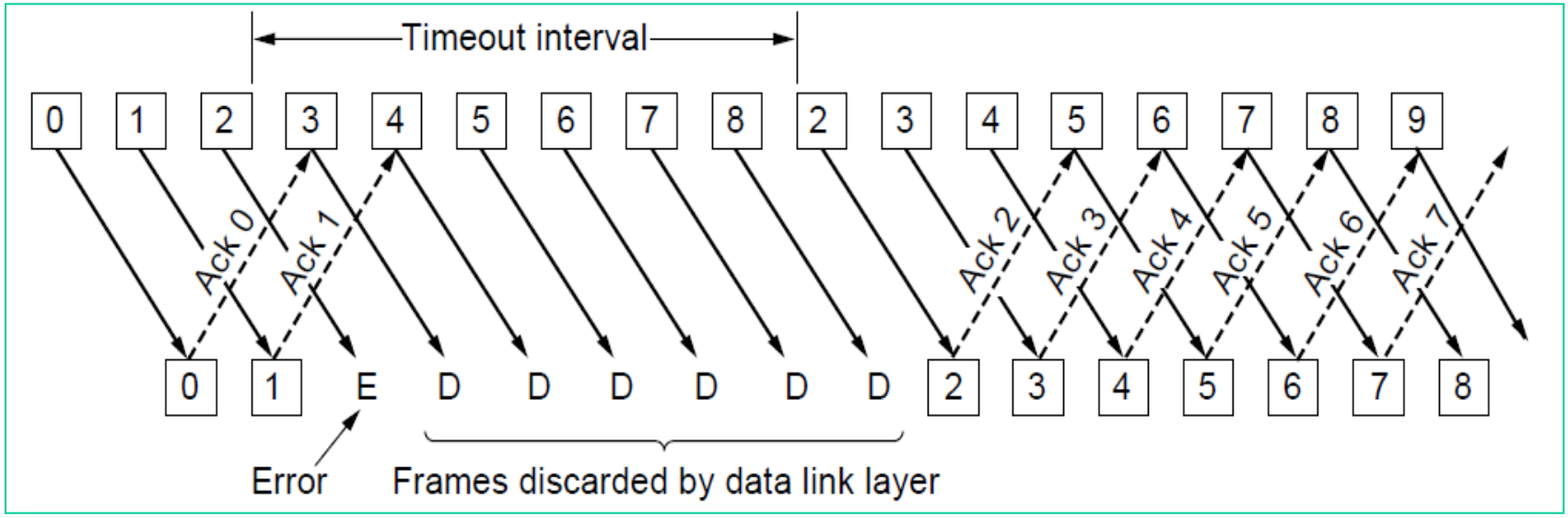
Pipelining

- In all previous protocols we have not considered channels with long Round Trip Time (RTT).
- RTT = Data frame propagation delay from sender to receiver + ack frame propagation delay from receiver to sender .
- RTT = 2 * Tp



A Protocol Using Go Back N

- In Go Back N approach the receiver's window size is 1. The figure below shows the working of Go Back N approach



A Protocol Using Go Back N

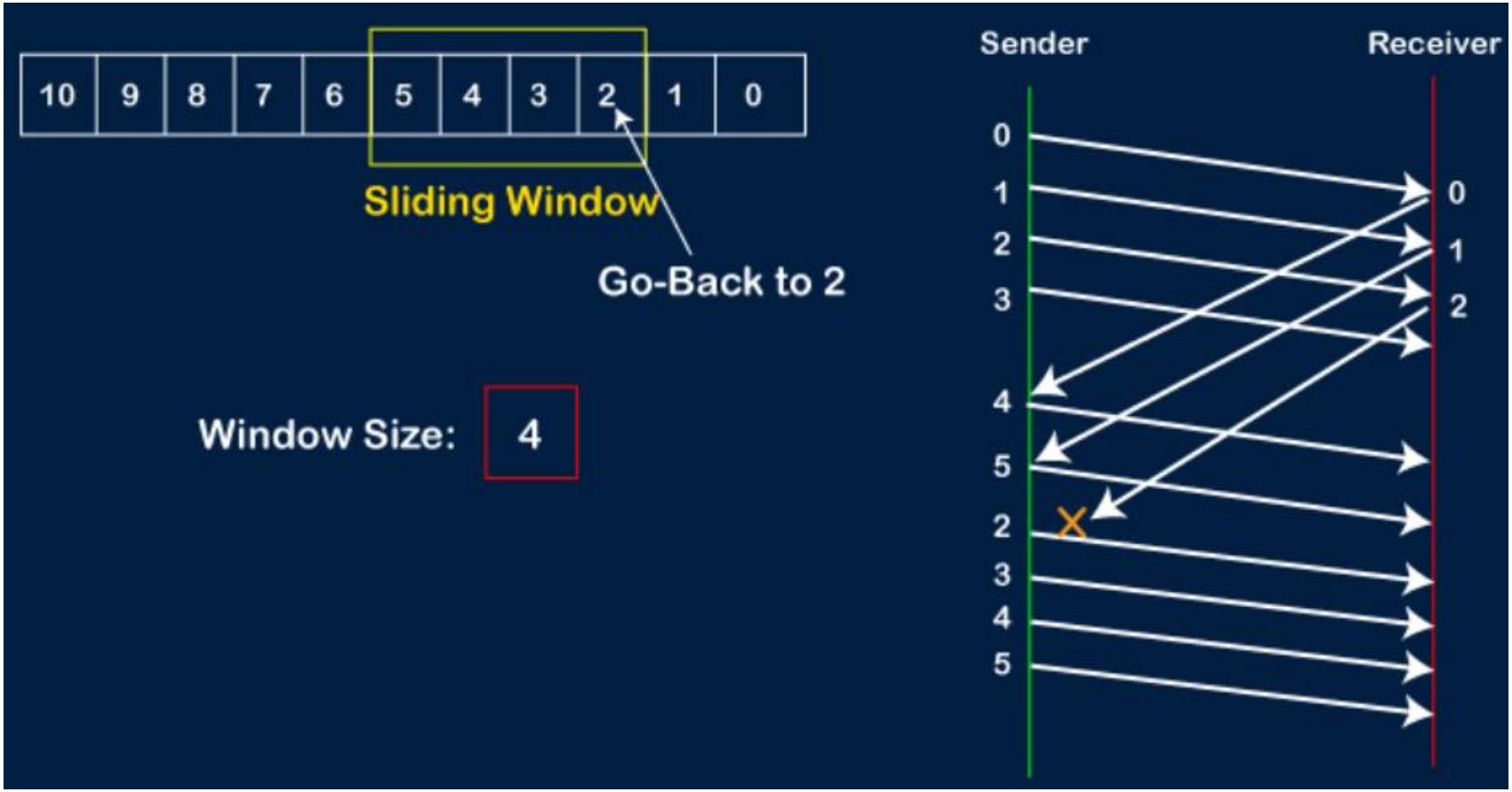
- In the Fig. given in the previous slide, we see the case in which the receiver's window is 1. Frames 0 and 1 are correctly received and acknowledged.

- Frame 2, however, **is damaged or lost**. The sender, unaware of this problem, continues to send frames until the timer for frame 2 expires.

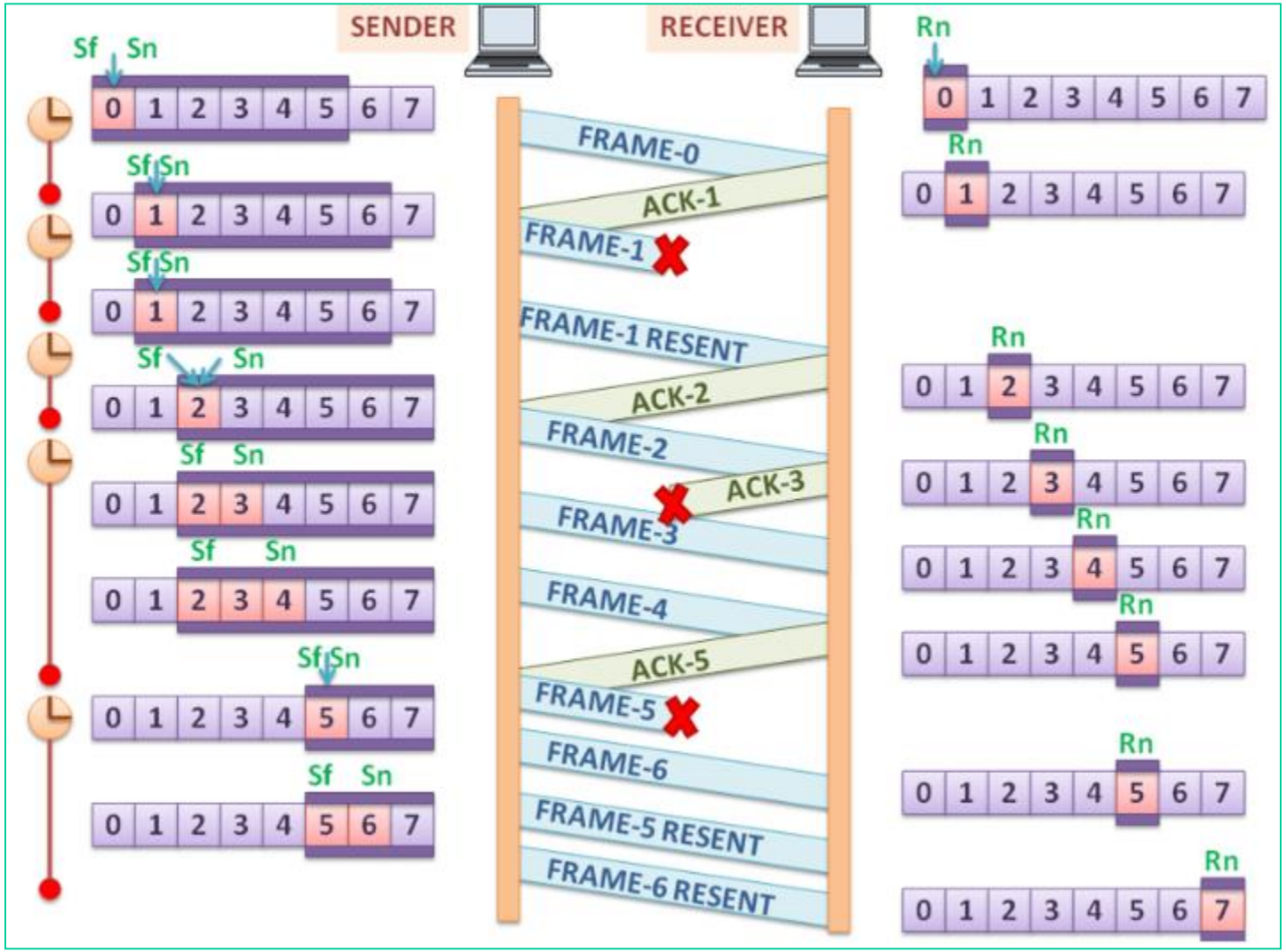
- Then it goes back to frame 2 and starts all over with it, sending 2, 3, 4, etc. all over again.

A Protocol Using Go Back N

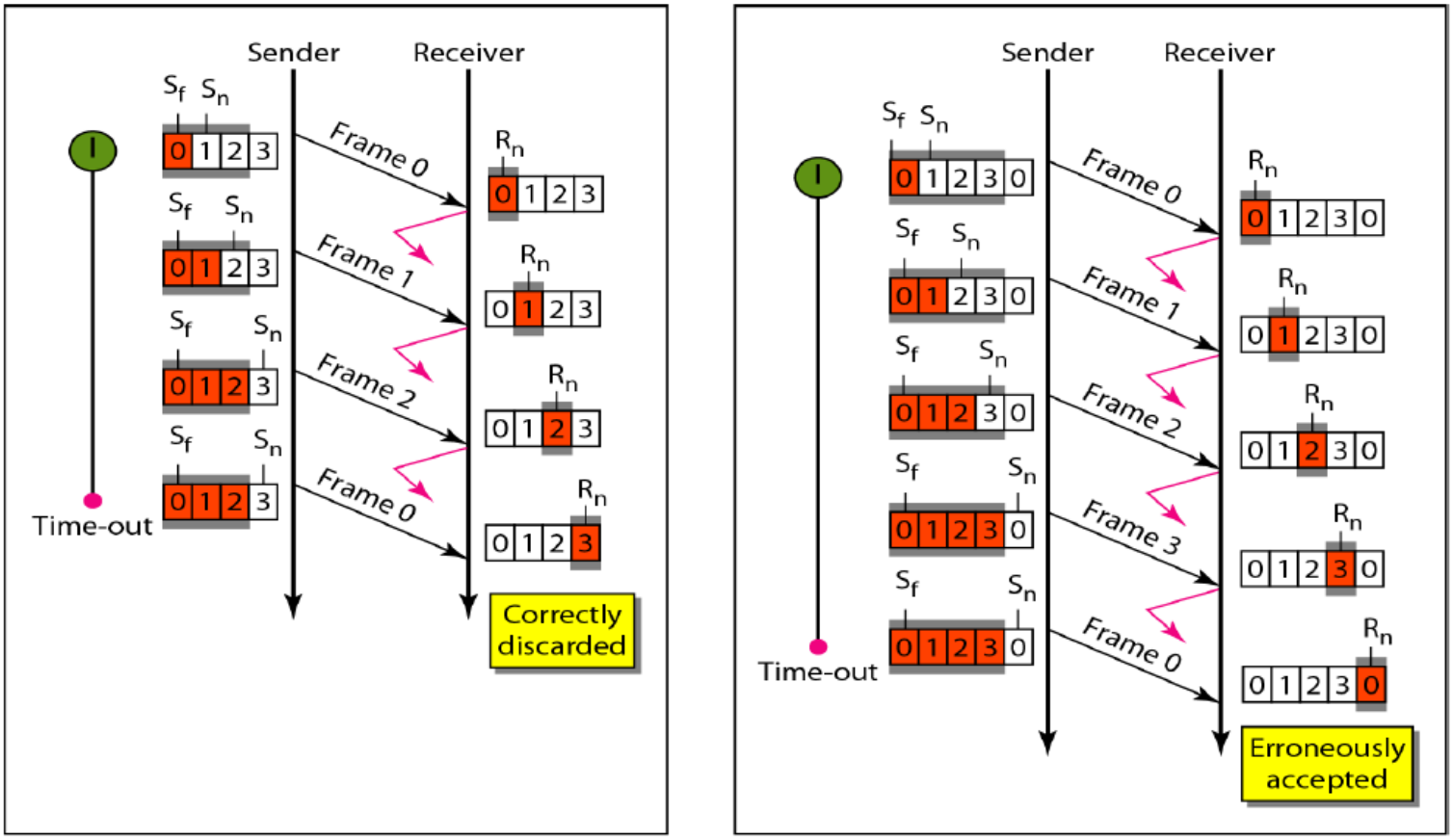
Its called Go Back N, because it has go back to nth damaged frame and start all over with it.



A Protocol Using Go Back N



A Protocol Using Go Back N



a. Window size $< 2^m$

b. Window size $= 2^m$

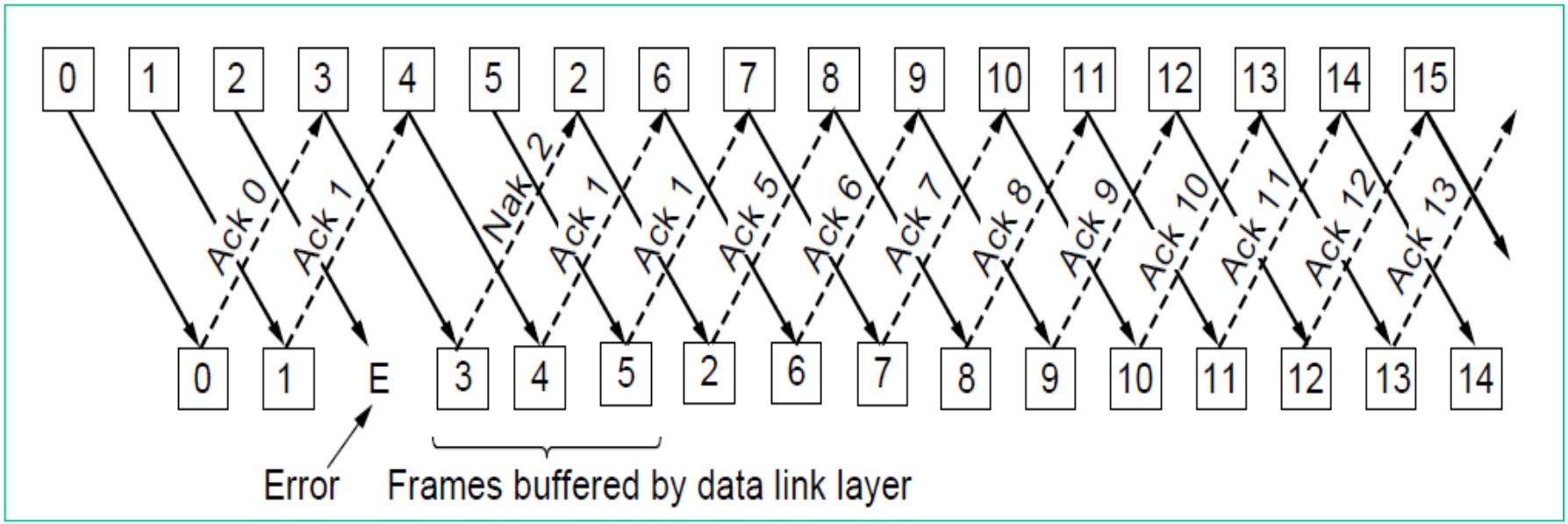
In Go-Back-N ARQ, the size of the send window must be less than 2^m; the size of the receiver window is always 1.

Selective Repeat

- Go Back N can waste a lot of bandwidth if the error rate is high.
- **Selective repeat uses window size more than 1 for the receiver.**
- So in Selective repeat a bad frame that is received is discarded, but good frames received after it are buffered.
- When the sender times out, only the oldest unacknowledged frame is retransmitted.

Selective Repeat

- If that frame arrives correctly, the receiver can deliver to the network layer, in sequence, all the frames it has buffered. The receiver sends a negative acknowledgement (NAK) when it detects an error.



Selective Repeat

Selective Repeat

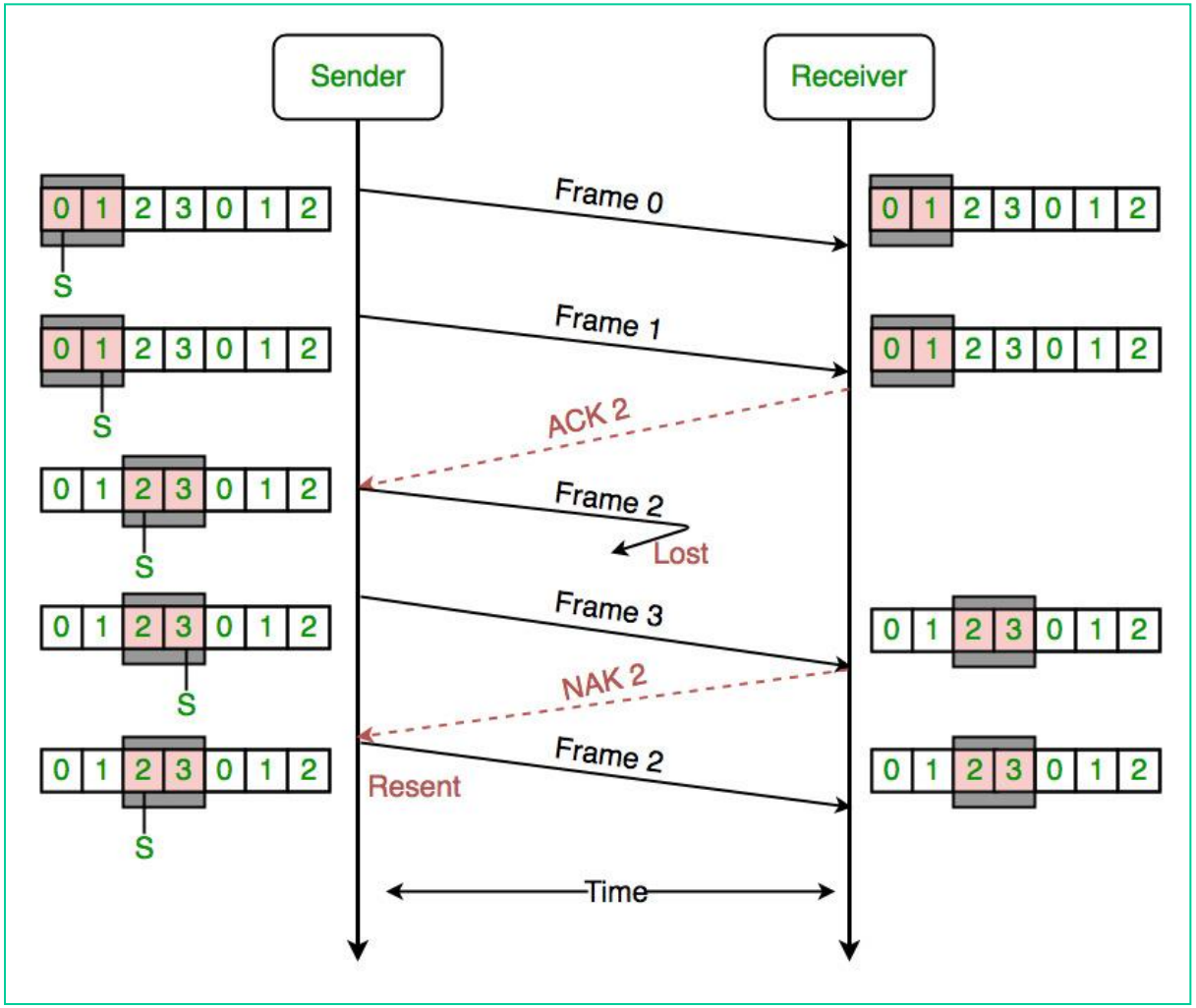
- In the Fig. 3-16, frames 0 and 1 are again correctly received and acknowledged and frame 2 is lost.
- When frame 3 arrives at the receiver, the data link layer there notices that is has missed a frame, so it sends back a NAK for 2 but buffers 3.
- When frames 4 and 5 arrive, they, too, are buffered by the data link layer instead of being passed to the network layer.
- Eventually, the NAK 2 gets back to the sender, which immediately resends frame 2. When that arrives, the data link layer now has 2, 3, 4, and 5 and can pass all of them to the network layer in the correct order.

Selective Repeat

- It can also acknowledge all frames up to and including 5, as shown in the figure.
- If the NAK should get lost, eventually the sender will time out for frame 2 and send it (and only it) of its own accord, but that may be a quite a while later.
- **In effect, the NAK speeds up the retransmission of one specific frame.**
- **Note that sender and receiver have the same window size.**

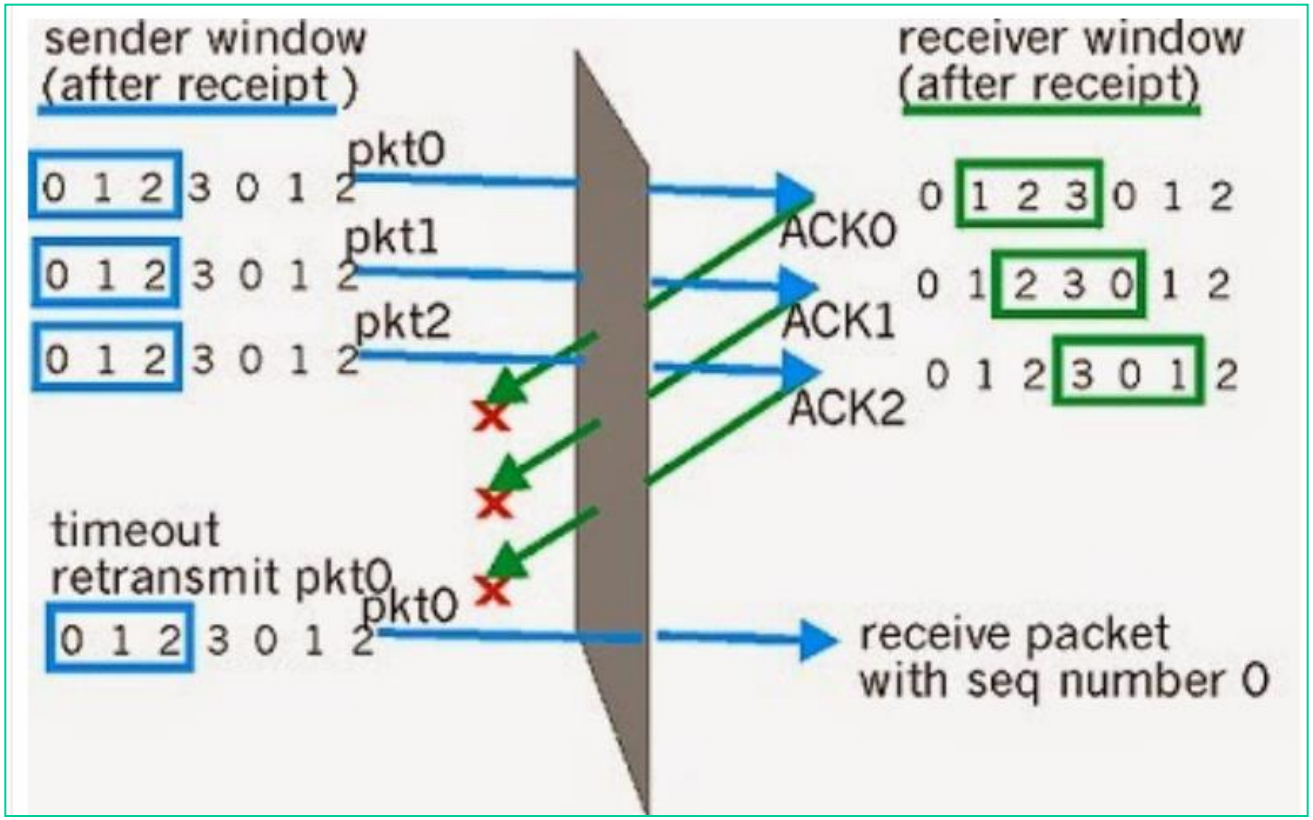
Selective Repeat

Example-1



Selective Repeat

How does the protocol behave in the following scenario ?



Selective Repeat

- The receiver maintains a window of sequence numbers for frames it is willing to accept. When a frame outside this window arrives, the receiver sends a duplicate ACK for the last successfully received in-order frame.
- This helps the sender identify which frames need to be retransmitted, improving efficiency compared to Go-Back-N which requires retransmission of all frames after a loss.
- The ability to send duplicate ACKs is a key advantage of Selective Repeat over simpler schemes, as it enables more selective and efficient retransmission of lost or corrupted data.

Differences

PROTOCOL:-	GO-BACK-N		SELECTIVE REPEAT
Bandwidth utilization	Medium		High
Maximum sender Size Window	2^m-1		2^(m-1)
Maximum receiver Size Window	1		2^(m-1)
Pipelining	Implemented		Implemented
Out of order Frames	Discarded		Accepted
Cumulative ACK	Applicable		Applicable